

LoRA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS

Edward Hu* Yelong Shen* Phillip Wallis Zeyuan Allen-Zhu
 Yuanzhi Li Shean Wang Lu Wang Weizhu Chen
 Microsoft Corporation
 {edwardhu, yeshe, phwallis, zeyuana,
 yuanzhil, swang, luw, wzchen}@microsoft.com
 yuanzhil@andrew.cmu.edu
 (Version 2)

ABSTRACT

An important paradigm of natural language processing consists of large-scale pre-training on general domain data and adaptation to particular tasks or domains. As we pre-train larger models, full fine-tuning, which re-trains all model parameters, becomes less feasible. Using GPT-3 175B as an example – deploying independent instances of fine-tuned models, each with 175B parameters, is prohibitively expensive. We propose **Low-Rank Adaptation**, or LoRA, which freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture, greatly reducing the number of trainable parameters for downstream tasks. Compared to GPT-3 175B fine-tuned with Adam, LoRA can reduce the number of trainable parameters by 10,000 times and the GPU memory requirement by 3 times. LoRA performs on-par or better than fine-tuning in model quality on RoBERTa, DeBERTa, GPT-2, and GPT-3, despite having fewer trainable parameters, a higher training throughput, and, unlike adapters, *no additional inference latency*. We also provide an empirical investigation into rank-deficiency in language model adaptation, which sheds light on the efficacy of LoRA. We release a package that facilitates the integration of LoRA with PyTorch models and provide our implementations and model checkpoints for RoBERTa, DeBERTa, and GPT-2 at <https://github.com/microsoft/LoRA>.

1 INTRODUCTION

Many applications in natural language processing rely on adapting *one* large-scale, pre-trained language model to *multiple* downstream applications. Such adaptation is usually done via *fine-tuning*, which updates all the parameters of the pre-trained model. The major downside of fine-tuning is that the new model contains as many parameters as in the original model. As larger models are trained every few months, this changes from a mere “inconvenience” for GPT-2 (Radford et al., b) or RoBERTa large (Liu et al., 2019) to a critical deployment challenge for GPT-3 (Brown et al., 2020) with 175 billion trainable parameters.¹

Many sought to mitigate this by adapting only some parameters or learning external modules for new tasks. This way, we only need to store and load a small number of task-specific parameters in addition to the pre-trained model for each task, greatly boosting the operational efficiency when deployed. However, existing techniques

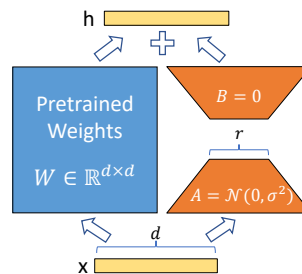


Figure 1: Our reparametrization. We only train A and B .

*Equal contribution.

⁰Compared to V1, this draft includes better baselines, experiments on GLUE, and more on adapter latency.

¹While GPT-3 175B achieves non-trivial performance with few-shot learning, fine-tuning boosts its performance significantly as shown in Appendix A.

LORA: 大型语言模型的低秩适配

胡爱德* 沈业龙* 菲利普·沃利斯 朱泽元 李元之 王善 王璐 陈伟烛

微软公司 {edwardhu, yeshe, phwallis, zeyuana, yuanzhil, swan
g, luw, wzchen}@microsoft.com yuanzhil@andrew.cmu.edu (版本 2)

摘要

自然语言处理的一个重要范式包括对通用领域数据进行大规模预训练以及对特定任务或领域进行适应。当我们预训练更大的模型时，全量微调（重新训练所有模型参数）变得不太可行。以 GPT-3 175B 为例——部署独立的微调模型实例，每个实例有 175B 个参数，成本高得难以承受。我们提出了低秩适配（Low-Rank Adaptation, LoRA），它冻结预训练模型的权重，并在 Transformer 架构的每一层中注入可训练的秩分解矩阵，大大减少下游任务的可训练参数数量。与使用 Adam 微调的 GPT-3 175B 相比，LoRA 可以将可训练参数数量减少 10,000 倍，GPU 内存需求降低 3 倍。尽管可训练参数更少、训练吞吐量更高，LoRA 在 RoBERTa、DeBERTa、GPT-2 和 GPT-3 上的模型质量与微调持平或更好。

1 引言

自然语言处理中的许多应用依赖于将大规模预训练语言模型适配到下游应用。这种适配通常通过微调来完成，微调会更新预训练模型的所有参数。微调的主要缺点是新模型包含与原模型相同数量的参数。随着每隔几个月就会训练出更大的模型，这对于 GPT-2 (Radford 等人, b) 或 RoBERTa large (Liu 等人, 2019) 来说只是一个“小不便”，但对于拥有 1750 亿可训练参数的 GPT-3 (Brown 等人, 2020) 来说，则成为一个关键的部署挑战。

许多人试图通过仅调整某些参数或为新任务学习外部模块来缓解这一问题。通过这种方式，我们只需要在每个任务中除了预训练模型外，存储和加载少量特定任务的参数，从而在部署时大大提高操作效率。然而，现有技术

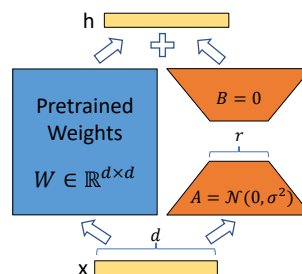


图1: 我们的重新参数化。我们只训练 A 和 B。

*Equal contribution.

⁰Compared to V1, this draft includes better baselines, experiments on GLUE, and more on adapter latency.

¹While GPT-3 175B achieves non-trivial performance with few-shot learning, fine-tuning boosts its performance significantly as shown in Appendix A.

often introduce inference latency (Houlsby et al., 2019; Rebuffi et al., 2017) by extending model depth or reduce the model’s usable sequence length (Li & Liang, 2021; Lester et al., 2021; Ham-bardzumyan et al., 2020; Liu et al., 2021) (Section 3). More importantly, these method often fail to match the fine-tuning baselines, posing a trade-off between efficiency and model quality.

We take inspiration from Li et al. (2018a); Aghajanyan et al. (2020) which show that the learned over-parametrized models in fact reside on a low intrinsic dimension. We hypothesize that the change in weights during model adaptation also has a low “intrinsic rank”, leading to our proposed **Low-Rank Adaptation (LoRA)** approach. LoRA allows us to train some dense layers in a neural network indirectly by optimizing rank decomposition matrices of the dense layers’ change during adaptation instead, while keeping the pre-trained weights frozen, as shown in Figure 1. Using GPT-3 175B as an example, we show that a very low rank (i.e., r in Figure 1 can be one or two) suffices even when the full rank (i.e., d) is as high as 12,288, making LoRA both storage- and compute-efficient.

LoRA possesses several key advantages.

- A pre-trained model can be shared and used to build many small LoRA modules for different tasks. We can freeze the shared model and efficiently switch tasks by replacing the matrices A and B in Figure 1, reducing the storage requirement and task-switching overhead significantly.
- LoRA makes training more efficient and lowers the hardware barrier to entry by up to 3 times when using adaptive optimizers since we do not need to calculate the gradients or maintain the optimizer states for most parameters. Instead, we only optimize the injected, much smaller low-rank matrices.
- Our simple linear design allows us to merge the trainable matrices with the frozen weights when deployed, *introducing no inference latency* compared to a fully fine-tuned model, by construction.
- LoRA is orthogonal to many prior methods and can be combined with many of them, such as prefix-tuning. We provide an example in Appendix E.

Terminologies and Conventions We make frequent references to the Transformer architecture and use the conventional terminologies for its dimensions. We call the input and output dimension size of a Transformer layer d_{model} . We use W_q , W_k , W_v , and W_o to refer to the query/key/value/output projection matrices in the self-attention module. W or W_0 refers to a pre-trained weight matrix and ΔW its accumulated gradient update during adaptation. We use r to denote the rank of a LoRA module. We follow the conventions set out by (Vaswani et al., 2017; Brown et al., 2020) and use Adam (Loshchilov & Hutter, 2019; Kingma & Ba, 2017) for model optimization and use a Transformer MLP feedforward dimension $d_{ffn} = 4 \times d_{model}$.

2 PROBLEM STATEMENT

While our proposal is agnostic to training objective, we focus on language modeling as our motivating use case. Below is a brief description of the language modeling problem and, in particular, the maximization of conditional probabilities given a task-specific prompt.

Suppose we are given a pre-trained autoregressive language model $P_\Phi(y|x)$ parametrized by Φ . For instance, $P_\Phi(y|x)$ can be a generic multi-task learner such as GPT (Radford et al., b; Brown et al., 2020) based on the Transformer architecture (Vaswani et al., 2017). Consider adapting this pre-trained model to downstream conditional text generation tasks, such as summarization, machine reading comprehension (MRC), and natural language to SQL (NL2SQL). Each downstream task is represented by a training dataset of context-target pairs: $\mathcal{Z} = \{(x_i, y_i)\}_{i=1, \dots, N}$, where both x_i and y_i are sequences of tokens. For example, in NL2SQL, x_i is a natural language query and y_i its corresponding SQL command; for summarization, x_i is the content of an article and y_i its summary.

通常会通过增加模型深度引入推理延迟 (Houlsby 等, 2019; Rebuffi 等, 2017), 或者减少模型可用的序列长度 (Li & Liang, 2021; Lester 等, 2021; Hambardzumyan 等, 2020; Liu 等, 2021) (第 3 节)。更重要的是, 这些方法通常无法匹配微调基线, 在效率与模型质量之间存在权衡。

我们从 Li 等人 (2018a) 和 Aghajanyan 等人 (2020) 的研究中获得灵感, 这些研究表明, 学习到的过参数化模型实际上存在于一个低内在维度中。我们假设, 在模型适应过程中权重的变化也具有低“内在秩”, 由此提出了我们的低秩适应 (LoRA) 方法。LoRA 允许我们通过优化适应过程中稠密层变化的秩分解矩阵来间接训练神经网络中的一些稠密层, 同时保持预训练权重不变, 如图 1 所示。以 GPT-3 175B 为例, 我们展示了即使全秩 (即图 1 中的 d) 高达 12,288, 极低秩 (即 r 可以是 1 或 2) 也足够, 使得 LoRA 在存储和计算上都非常高效。

LoRA 拥有几个关键优势。

- 一个预训练模型可以被共享并用于构建许多针对不同任务的小型 LoRA 模块。我们可以冻结共享模型, 并通过替换图 1 中的矩阵 A 和 B 来高效地切换任务, 从而显著减少存储需求和任务切换开销。
- LoRA 在使用自适应优化器时, 使训练更加高效, 并将硬件门槛降低了最多三倍, 因为我们不需要为大多数参数计算梯度或维护优化器状态。相反, 我们只优化注入的、要小得多的低秩矩阵。
- 我们简单的线性设计允许我们在部署时将可训练矩阵与冻结权重合并, *introducing no inference latency*, 与完全微调的模型相比, 这是结构上本来的设计。
- LoRA 与许多先前的方法是正交的, 并且可以与其中的许多方法结合, 例如前缀调优。我们在附录 E 中提供了一个示例。

术语和惯例 我们经常提到 Transformer 架构, 并使用其维度的常规术语。我们将 Transformer 层的输入和输出维度大小称为 d_{model} 。我们使用 W_q 、 W_k 、 W_v 和 W_o 来指代自注意力模块中的 query/key/value/output 投影矩阵。 W 或 W_0 指预训练权重矩阵, ΔW 指其在适应过程中累积的梯度更新。我们使用 r 来表示 LoRA 模块的秩。我们遵循 (Vaswani et al., 2017; Brown et al., 2020) 的惯例, 并使用 Adam (Loshchilov & Hutter, 2019; Kingma & Ba, 2017) 进行模型优化, 并使用 Transformer MLP 前馈维度 $d_{ffn} = \text{乘} \times d_{model}$

2 问题陈述

虽然我们的提议对训练目标持中立态度, 但我们将语言建模作为我们的主要应用案例。下面是对语言建模问题的简要描述, 特别是针对特定任务提示最大化条件概率的内容。

假设我们得到一个预训练的自回归语言模型 $P_\Phi(y|x)$, 其参数为 Φ 。例如, $P_\Phi(y|x)$ 可以是一个通用的多任务学习器, 如基于 Transformer 架构的 GPT (Radford 等人, b; Brown 等人, 2020) (Vaswani 等人, 2017)。考虑将这个预训练模型适应到下游条件文本生成任务, 例如摘要生成、机器阅读理解 (MRC) 和自然语言到 SQL (NL2SQL)。每个下游任务由一个上下文-目标对的训练数据集表示: $\mathcal{Z} = \{(x_i, y_i)\}_{i=1, \dots, N}$, 其中 x_i 和 y_i 都是标记序列。例如, 在 NL2SQL 中, x_i 是自然语言查询, y_i 是其对应的 SQL 命令; 在摘要生成中, x_i 是文章内容, y_i 是其摘要。

During full fine-tuning, the model is initialized to pre-trained weights Φ_0 and updated to $\Phi_0 + \Delta\Phi$ by repeatedly following the gradient to maximize the conditional language modeling objective:

$$\max_{\Phi} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log(P_{\Phi}(y_t|x, y_{<t})) \quad (1)$$

One of the main drawbacks for full fine-tuning is that for *each* downstream task, we learn a *different* set of parameters $\Delta\Phi$ whose dimension $|\Delta\Phi|$ equals $|\Phi_0|$. Thus, if the pre-trained model is large (such as GPT-3 with $|\Phi_0| \approx 175$ Billion), storing and deploying many independent instances of fine-tuned models can be challenging, if at all feasible.

In this paper, we adopt a more parameter-efficient approach, where the task-specific parameter increment $\Delta\Phi = \Delta\Phi(\Theta)$ is further encoded by a much smaller-sized set of parameters Θ with $|\Theta| \ll |\Phi_0|$. The task of finding $\Delta\Phi$ thus becomes optimizing over Θ :

$$\max_{\Theta} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log(p_{\Phi_0 + \Delta\Phi(\Theta)}(y_t|x, y_{<t})) \quad (2)$$

In the subsequent sections, we propose to use a low-rank representation to encode $\Delta\Phi$ that is both compute- and memory-efficient. When the pre-trained model is GPT-3 175B, the number of trainable parameters $|\Theta|$ can be as small as 0.01% of $|\Phi_0|$.

3 AREN'T EXISTING SOLUTIONS GOOD ENOUGH?

The problem we set out to tackle is by no means new. Since the inception of transfer learning, dozens of works have sought to make model adaptation more parameter- and compute-efficient. See Section 6 for a survey of some of the well-known works. Using language modeling as an example, there are two prominent strategies when it comes to efficient adaptations: adding adapter layers (Houlsby et al., 2019; Rebuffi et al., 2017; Pfeiffer et al., 2021; Rücklé et al., 2020) or optimizing some forms of the input layer activations (Li & Liang, 2021; Lester et al., 2021; Hambardzumyan et al., 2020; Liu et al., 2021). However, both strategies have their limitations, especially in a large-scale and latency-sensitive production scenario.

Adapter Layers Introduce Inference Latency There are many variants of adapters. We focus on the original design by Houlsby et al. (2019) which has two adapter layers per Transformer block and a more recent one by Lin et al. (2020) which has only one per block but with an additional LayerNorm (Ba et al., 2016). While one can reduce the overall latency by pruning layers or exploiting multi-task settings (Rücklé et al., 2020; Pfeiffer et al., 2021), there is no direct ways to bypass the extra compute in adapter layers. This seems like a non-issue since adapter layers are designed to have few parameters (sometimes <1% of the original model) by having a small bottleneck dimension, which limits the FLOPs they can add. However, large neural networks rely on hardware parallelism to keep the latency low, and adapter layers have to be processed sequentially. This makes a difference in the online inference setting where the batch size is typically as small as one. In a generic scenario without model parallelism, such as running inference on GPT-2 (Radford et al., b) medium on a single GPU, we see a noticeable increase in latency when using adapters, even with a very small bottleneck dimension (Table 1).

This problem gets worse when we need to shard the model as done in Shoeybi et al. (2020); Lepikhin et al. (2020), because the additional depth requires more synchronous GPU operations such as `AllReduce` and `Broadcast`, unless we store the adapter parameters redundantly many times.

Directly Optimizing the Prompt is Hard The other direction, as exemplified by prefix tuning (Li & Liang, 2021), faces a different challenge. We observe that prefix tuning is difficult to optimize and that its performance changes non-monotonically in trainable parameters, confirming similar observations in the original paper. More fundamentally, reserving a part of the sequence length for adaptation necessarily reduces the sequence length available to process a downstream task, which we suspect makes tuning the prompt less performant compared to other methods. We defer the study on task performance to Section 5.

在完全微调期间，模型初始化为预训练权重 Φ_0 ，并通过反复按照梯度更新以最大化条件语言建模目标，更新到 $\Phi_0 + \Delta\Phi$ ：

$$\max_{\Phi} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log(P_{\Phi}(y_t|x, y_{<t})) \quad (1)$$

完全微调的主要缺点之一是，对于 *each* 下游任务，我们需要学习一组 *different* 参数 $\Delta\Phi$ ，其维度 $|\Delta\Phi|$ 等于 $|\Phi_0|$ 。因此，如果预训练模型很大（例如具有 $|\Phi_0| \approx 1750$ 亿参数的 GPT-3），存储和部署许多独立的微调模型实例可能会非常具有挑战性，甚至可能不可行。

在本文中，我们采用了一种更参数高效的方法，其中任务特定的参数增量 $\Delta\Phi = \Delta\Phi(\Theta)$ 进一步由一组更小尺寸的参数 Θ 与 $|\Theta| \ll |\Phi_0|$ 编码。因此，寻找 $\Delta\Phi$ 的任务变为在 Θ 上进行优化：

$$\max_{\Theta} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log(p_{\Phi_0 + \Delta\Phi(\Theta)}(y_t|x, y_{<t})) \quad (2)$$

在随后的章节中，我们提出使用低秩表示来编码 $\Delta\Phi$ ，这种表示在计算和内存上都是高效的。当预训练模型为 GPT-3 175B 时，可训练参数的数量 $|\Theta|$ 最小可以达到 $|\Phi_0|$ 的 0.01%。

3 现有的解决方案不够好吗？

我们着手解决的问题绝不是新问题。自迁移学习诞生以来，几十项工作致力于使模型适应在参数和计算上更加高效。有关一些知名工作的综述，请参见第6节。以语言建模为例，在高效适应方面有两种显著的策略：添加适配器层（Houlsby 等，2019；Rebuffi 等，2017；Pfeiffer 等，2021；Rücklé 等，2020）或优化某些形式的输入层激活（Li & Liang, 2021；Lester 等，2021；Hambardzumyan 等，2020；Liu 等，2021）。然而，这两种策略都有其局限性，尤其是在大规模和对延迟敏感的生产场景中。

适配器层引入推理延迟 适配器有许多变体。我们关注 Houlsby 等人（2019）提出的原始设计，该设计在每个 Transformer 块中有两个适配器层，以及 Lin 等人（2020）提出的一个较新的设计，该设计每个块仅有一个适配器层，但附加了一个 LayerNorm（Ba 等人，2016）。虽然可以通过剪枝层或利用多任务设置来减少整体延迟（Rücklücke 等人，2020；Pfeiffer 等人，2021），但没有直接的方法可以绕过适配器层中的额外计算。这似乎不是问题，因为适配器层设计为参数较少（有时仅为原始模型的 1%），通过较小的瓶颈维度限制它们能够增加的 FLOPs。然而，大型神经网络依赖硬件并行性来保持低延迟，而适配器层必须依次处理。这在在线推理设置中会产生区别，因为批量大小通常小到只有一个。在通用场

当我们需要像 Shoeybi 等（2020）和 Lepikhin 等（2020）所做的那样对模型进行分片时，这个问题会变得更严重，因为额外的深度需要更多的同步 GPU 操作，例如 AllReduce 和 Broadcast，除非我们将适配器参数冗余存储多次。

直接优化提示很困难。另一种方式，如前缀调优（Li & Liang, 2021）所示，则面临不同的挑战。我们观察到前缀调优很难优化，并且其性能在可训练参数上呈非单调变化，这与原论文的类似观察结果一致。更根本的是，为适应保留序列长度的一部分，必然会减少可用于处理下游任务的序列长度，我们怀疑这使得提示调优的性能相比其他方法较低。我们将任务性能的研究延迟到第5节讨论。

Batch Size	32	16	1
Sequence Length	512	256	128
$ \Theta $	0.5M	11M	11M
Fine-Tune/LoRA	1449.4 $\pm$ 0.8	338.0 $\pm$ 0.6	19.8 $\pm$ 2.7
Adapter ^L	1482.0 $\pm$ 1.0 (+2.2%)	354.8 $\pm$ 0.5 (+5.0%)	23.9 $\pm$ 2.1 (+20.7%)
Adapter ^H	1492.2 $\pm$ 1.0 (+3.0%)	366.3 $\pm$ 0.5 (+8.4%)	25.8 $\pm$ 2.2 (+30.3%)

Table 1: Inference latency of a single forward pass in GPT-2 medium measured in milliseconds, averaged over 100 trials. We use an NVIDIA Quadro RTX8000. “ $|\Theta|$ ” denotes the number of trainable parameters in adapter layers. Adapter^L and Adapter^H are two variants of adapter tuning, which we describe in Section 5.1. The inference latency introduced by adapter layers can be significant in an online, short-sequence-length scenario. See the full study in Appendix B.

4 OUR METHOD

We describe the simple design of LoRA and its practical benefits. The principles outlined here apply to any dense layers in deep learning models, though we only focus on certain weights in Transformer language models in our experiments as the motivating use case.

4.1 LOW-RANK-PARAMETRIZED UPDATE MATRICES

A neural network contains many dense layers which perform matrix multiplication. The weight matrices in these layers typically have full-rank. When adapting to a specific task, Aghajanyan et al. (2020) shows that the pre-trained language models have a low “intrinsic dimension” and can still learn efficiently despite a random projection to a smaller subspace. Inspired by this, we hypothesize the updates to the weights also have a low “intrinsic rank” during adaptation. For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, we constrain its update by representing the latter with a low-rank decomposition $W_0 + \Delta W = W_0 + BA$, where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and the rank $r \ll \min(d, k)$. During training, W_0 is frozen and does not receive gradient updates, while A and B contain trainable parameters. Note both W_0 and $\Delta W = BA$ are multiplied with the same input, and their respective output vectors are summed coordinate-wise. For $h = W_0x$, our modified forward pass yields:

$$h = W_0x + \Delta Wx = W_0x + BAx \quad (3)$$

We illustrate our reparametrization in Figure 1. We use a random Gaussian initialization for A and zero for B , so $\Delta W = BA$ is zero at the beginning of training. We then scale ΔWx by $\frac{\alpha}{r}$, where α is a constant in r . When optimizing with Adam, tuning α is roughly the same as tuning the learning rate if we scale the initialization appropriately. As a result, we simply set α to the first r we try and do not tune it. This scaling helps to reduce the need to retune hyperparameters when we vary r (Yang & Hu, 2021).

A Generalization of Full Fine-tuning. A more general form of fine-tuning allows the training of a subset of the pre-trained parameters. LoRA takes a step further and does not require the accumulated gradient update to weight matrices to have full-rank during adaptation. This means that when applying LoRA to all weight matrices and training all biases², we roughly recover the expressiveness of full fine-tuning by setting the LoRA rank r to the rank of the pre-trained weight matrices. In other words, as we increase the number of trainable parameters³, training LoRA roughly converges to training the original model, while adapter-based methods converges to an MLP and prefix-based methods to a model that cannot take long input sequences.

No Additional Inference Latency. When deployed in production, we can explicitly compute and store $W = W_0 + BA$ and perform inference as usual. Note that both W_0 and BA are in $\mathbb{R}^{d \times k}$. When we need to switch to another downstream task, we can recover W_0 by subtracting BA and then adding a different $B'A'$, a quick operation with very little memory overhead. Critically, this

²They represent a negligible number of parameters compared to weights.

³An inevitability when adapting to hard tasks.

Batch Size	32	16	1
Sequence Length	512	256	128
$ \Theta $	0.5M	11M	11M
Fine-Tune/LoRA	1449.4 $\pm$ 0.8	338.0 $\pm$ 0.6	19.8 $\pm$ 2.7
Adapter ^L	1482.0 $\pm$ 1.0 (+2.2%)	354.8 $\pm$ 0.5 (+5.0%)	23.9 $\pm$ 2.1 (+20.7%)
Adapter ^H	1492.2 $\pm$ 1.0 (+3.0%)	366.3 $\pm$ 0.5 (+8.4%)	25.8 $\pm$ 2.2 (+30.3%)

表 1: GPT-2 中型模型单次前向传播的推理延迟，以毫秒为单位测量，并在 100 次试验中取平均。我们使用 NVIDIA Quadro RTX8000。“ $|\Theta|$ ”表示适配器层中可训练参数的数量。Adapter^L 和 Adapter^H 是适配器调优的两种变体，我们在第 5.1 节中进行了描述。在在线、短序列长度的场景中，适配器层引入的推理延迟可能相当显著。完整研究见附录 B。

4 我们的方法

我们描述了 LoRA 的简单设计及其实际益处。这里概述的原则适用于深度学习模型中的任何密集层，尽管在我们的实验中，我们仅关注 Transformer 语言模型中的某些权重，作为激励使用案例。

4.1 低秩参数化更新矩阵

一个神经网络包含许多执行矩阵乘法的全连接层。这些层中的权重矩阵通常是满秩的。在适应特定任务时，Aghajanyan 等人 (2020) 显示预训练的语言模型具有低“内在维度”，即使随机投影到一个较小的子空间，也仍然能够高效学习。受到此启发，我们假设权重在适应过程中更新时也具有低“内在秩”。对于一个预训练的权重矩阵 $W_0 \in \mathbb{R}^{d \times k}$ ，我们使用低秩分解 $W_0 + \Delta W = W_0 + BA$ 来约束其更新，其中 $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ ，并且秩 $r \ll \min(d, k)$ 。在训练过程中， W_0 被固定，不接收梯度更新，而 A 和 B 包含可训练的参数。注意 W_0 和 $\Delta W = BA$ 都与相同的输入相乘，它们各自的输出向量按坐标相加。对于 $h = W_0 x$ ，我们修改后的前向传播得到：

$$h = W_0 x + \Delta W x = W_0 x + BAx \quad (3)$$

我们在图 1 中说明了我们的重参数化。我们对 A 使用随机高斯初始化，对 B 使用零初始化，因此 $\Delta W = BA$ 在训练开始时为零。然后我们通过 $\frac{\alpha}{r}$ 对 $\Delta W x$ 进行缩放，其中 α 是 r 中的一个常数。在使用 Adam 进行优化时，如果我们适当缩放初始化，调整 α 大致等同于调整学习率。因此，我们只是将 α 设置为我们尝试的第一个 r ，并且不进行调整。这种缩放有助于在我们改变 r 时减少重新调整超参数的需要 (Yang & Hu, 2021)。

全量微调的一个概括。微调的一种更通用形式允许对预训练参数的一部分进行训练。LoRA 更进一步，不要求在适应过程中权重矩阵的累积梯度更新具有满秩。这意味着，当将 LoRA 应用于所有权重矩阵并训练所有偏置时，通过将 LoRA 的秩设置为预训练权重矩阵的秩，我们大致可以恢复全量微调的表达能。换句话说，随着可训练参数数量的增加，训练 LoRA 大致会收敛到训练原始模型，而基于适配器的方法会收敛到一个 MLP，基于前缀的方法会收敛到一个无法处理长输入序列的模型。

没有额外的推理延迟。在生产环境中部署时，我们可以明确计算并存储 $W = W_0 + BA$ 并按照常规进行推理。请注意， W_0 和 BA 都在 $\mathbb{R}^{d \times k}$ 中。当我们需要切换到另一个下游任务时，我们可以通过减去 BA 然后加上一个不同的 $B'A'$ 来恢复 W_0 ，这是一个快速操作，内存开销很小。关键是，这

²They represent a negligible number of parameters compared to weights.

³An inevitability when adapting to hard tasks.

guarantees that we do not introduce any additional latency during inference compared to a fine-tuned model by construction.

4.2 APPLYING LORA TO TRANSFORMER

In principle, we can apply LoRA to any subset of weight matrices in a neural network to reduce the number of trainable parameters. In the Transformer architecture, there are four weight matrices in the self-attention module (W_q, W_k, W_v, W_o) and two in the MLP module. We treat W_q (or W_k, W_v) as a single matrix of dimension $d_{model} \times d_{model}$, even though the output dimension is usually sliced into attention heads. We limit our study to **only adapting the attention weights** for downstream tasks and freeze the MLP modules (so they are not trained in downstream tasks) both for simplicity and parameter-efficiency. We further study the effect on adapting different types of attention weight matrices in a Transformer in Section 7.1. We leave the empirical investigation of adapting the MLP layers, LayerNorm layers, and biases to a future work.

Practical Benefits and Limitations. The most significant benefit comes from the reduction in memory and storage usage. For a large Transformer trained with Adam, we reduce that VRAM usage by up to $2/3$ if $r \ll d_{model}$ as we do not need to store the optimizer states for the frozen parameters. On GPT-3 175B, we reduce the VRAM consumption during training from 1.2TB to 350GB. With $r = 4$ and only the query and value projection matrices being adapted, the checkpoint size is reduced by roughly $10,000\times$ (from 350GB to 35MB)⁴. This allows us to train with significantly fewer GPUs and avoid I/O bottlenecks. Another benefit is that we can switch between tasks while deployed at a much lower cost by only swapping the LoRA weights as opposed to all the parameters. This allows for the creation of many customized models that can be swapped in and out on the fly on machines that store the pre-trained weights in VRAM. We also observe a 25% speedup during training on GPT-3 175B compared to full fine-tuning⁵ as we do not need to calculate the gradient for the vast majority of the parameters.

LoRA also has its limitations. For example, it is not straightforward to batch inputs to different tasks with different A and B in a single forward pass, if one chooses to absorb A and B into W to eliminate additional inference latency. Though it is possible to not merge the weights and dynamically choose the LoRA modules to use for samples in a batch for scenarios where latency is not critical.

5 EMPIRICAL EXPERIMENTS

We evaluate the downstream task performance of LoRA on RoBERTa (Liu et al., 2019), DeBERTa (He et al., 2021), and GPT-2 (Radford et al., b), before scaling up to GPT-3 175B (Brown et al., 2020). Our experiments cover a wide range of tasks, from natural language understanding (NLU) to generation (NLG). Specifically, we evaluate on the GLUE (Wang et al., 2019) benchmark for RoBERTa and DeBERTa. We follow the setup of Li & Liang (2021) on GPT-2 for a direct comparison and add WikiSQL (Zhong et al., 2017) (NL to SQL queries) and SAMSum (Gliwa et al., 2019) (conversation summarization) for large-scale experiments on GPT-3. See Appendix C for more details on the datasets we use. We use NVIDIA Tesla V100 for all experiments.

5.1 BASELINES

To compare with other baselines broadly, we replicate the setups used by prior work and reuse their reported numbers whenever possible. This, however, means that some baselines might only appear in certain experiments.

Fine-Tuning (FT) is a common approach for adaptation. During fine-tuning, the model is initialized to the pre-trained weights and biases, and all model parameters undergo gradient updates. A simple variant is to update only some layers while freezing others. We include one such baseline reported in prior work (Li & Liang, 2021) on GPT-2, which adapts just the last two layers (**FT^{Top2}**).

⁴We still need the 350GB model during deployment; however, storing 100 adapted models only requires $350\text{GB} + 35\text{MB} * 100 \approx 354\text{GB}$ as opposed to $100 * 350\text{GB} \approx 35\text{TB}$.

⁵For GPT-3 175B, the training throughput for full fine-tuning is 32.5 tokens/s per V100 GPU; with the same number of weight shards for model parallelism, the throughput is 43.1 tokens/s per V100 GPU for LoRA.

通过构建保证，在推理过程中我们不会引入任何额外的延迟，相比于一个微调过的模型。

4.2 将 LoRA 应用于 Transformer

原则上，我们可以将 LoRA 应用于神经网络中任何一个权重矩阵子集，以减少可训练参数的数量。在 Transformer 架构中，自注意力模块有四个权重矩阵 (W_q, W_k, W_v, W_o)，MLP 模块有两个权重矩阵。我们将 W_q (或 W_k, W_v) 视为一个维度为 $d_{model} \times d_{model}$ 的单一矩阵，尽管输出维度通常会被切分成注意力头。为了简化操作和提高参数效率，我们的研究仅限于在下游任务中适配注意力权重，并冻结 MLP 模块（因此它们在下游任务中不进行训练）。我们在第 7.1 节进一步研究了适配 Transformer 中不同类型注意力权重矩阵的效果。对于适配 MLP 层、LayerNorm 层和偏置参数的实证研究，我们留待未来工作开展。

实际优势与限制。最显著的优势来自于内存和存储使用量的减少。对于使用 Adam 训练的大型 Transformer，我们可以将 VRAM 使用量最多减少 2/3，如果 $r \ll d_{model}$ ，因为我们不需要存储冻结参数的优化器状态。在 GPT-3 175B 上，我们将训练期间的 VRAM 消耗从 1.2TB 减少到 350GB。使用 $r = 4$ ，并且仅适应查询和值的投影矩阵，检查点大小大约从 350GB 减少到 35MB⁴，减少约 $10,000 \times$ 。这使我们能够使用显著更少的 GPU 进行训练，并避免 I/O 瓶颈。另一个好处是，在部署时，我们可以通过仅交换 LoRA 权重而不是所有参数，以更低的成本在任务之间切换。这允许创建许多自定义模型，并在存储预训练权重的机器上即时切换。我们还观察到，在 GPT-3 175B 上与完全微调相比，训练速度提升了 25%⁵。

LoRA 也有其局限性。例如，如果选择将 A 和 B 吸收到 W 中以消除额外的推理延迟，那么在单次前向传播中将不同任务的输入与不同的 A 和 B 批处理并不是很简单。尽管在延迟不关键的场景下，可以合并权重，并动态选择批处理中样本使用的 LoRA 模块。

5 个实证实验

我们评估了 LoRA 在 RoBERTa (Liu 等, 2019)、DeBERTa (He 等, 2021) 和 GPT-2 (Radford 等, 2019) 上的下游任务表现，然后再扩展到 GPT-3 175B (Brown 等, 2020)。我们的实验覆盖了广泛的任务，从自然语言理解 (NLU) 到生成 (NLG)。具体来说，我们在 RoBERTa 和 DeBERTa 上评估 GLUE (Wang 等, 2019) 基准。我们在 GPT-2 上遵循 Li & Liang (2021) 的设置以进行直接比较，并为 GPT-3 上的大规模实验添加了 WikiSQL (Zhong 等, 2017)（自然语言到 SQL 查询）和 SAMSum (Gliwa 等, 2019)（对话摘要）。更多关于我们使用的数据集的细节见附录 C。我们使用 NVIDIA Tesla V100 进行所有实验。

5.1 基线

为了与其他基线进行广泛比较，我们复制了先前工作的设置，并在可能的情况下重用他们报告的数字。然而，这也意味着某些基线可能只出现在特定的实验中。

微调 (FT) 是一种常见的适应方法。在微调过程中，模型初始化为预训练的权重和偏置，并且所有模型参数都会进行梯度更新。一种简单的变体是只更新部分层，而冻结其他层。我们包含了先前工作 (Li & Liang, 2021) 在 GPT-2 上报告的一个此类基线，该方法仅适应最后两层 (FT^{Top2})。

⁴We still need the 350GB model during deployment; however, storing 100 adapted models only requires 350GB + 35MB * 100 $\approx$ 354GB as opposed to 100 * 350GB $\approx$ 35TB.

⁵For GPT-3 175B, the training throughput for full fine-tuning is 32.5 tokens/s per V100 GPU; with the same number of weight shards for model parallelism, the throughput is 43.1 tokens/s per V100 GPU for LoRA.

Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB _{base} (FT)*	125.0M	87.6	94.8	90.2	63.6	92.8	91.9	78.7	91.2	86.4
RoB _{base} (BitFit)*	0.1M	84.7	93.7	92.7	62.0	91.8	84.0	81.5	90.8	85.2
RoB _{base} (Adpt ^D)*	0.3M	87.1 $\pm$ 0	94.2 $\pm$ 1	88.5 $\pm$ 1.1	60.8 $\pm$ 4	93.1 $\pm$ 1	90.2 $\pm$ 0	71.5 $\pm$ 2.7	89.7 $\pm$ 3	84.4
RoB _{base} (Adpt ^D)*	0.9M	87.3 $\pm$ 1	94.7 $\pm$ 3	88.4 $\pm$ 1	62.6 $\pm$ 9	93.0 $\pm$ 2	90.6 $\pm$ 0	75.9 $\pm$ 2.2	90.3 $\pm$ 1	85.4
RoB _{base} (LoRA)	0.3M	87.5 $\pm$ 3	95.1$\pm$2	89.7 $\pm$ 7	63.4 $\pm$ 1.2	93.3$\pm$3	90.8 $\pm$ 1	86.6$\pm$7	91.5$\pm$2	87.2
RoB _{large} (FT)*	355.0M	90.2	96.4	90.9	68.0	94.7	92.2	86.6	92.4	88.9
RoB _{large} (LoRA)	0.8M	90.6$\pm$2	96.2 $\pm$ 5	90.9$\pm$1.2	68.2$\pm$1.9	94.9$\pm$3	91.6 $\pm$ 1	87.4$\pm$2.5	92.6$\pm$2	89.0
RoB _{large} (Adpt ^P)†	3.0M	90.2 $\pm$ 3	96.1 $\pm$ 3	90.2 $\pm$ 7	68.3$\pm$1.0	94.8$\pm$2	91.9$\pm$1	83.8 $\pm$ 2.9	92.1 $\pm$ 7	88.4
RoB _{large} (Adpt ^P)†	0.8M	90.5$\pm$3	96.6$\pm$2	89.7 $\pm$ 1.2	67.8 $\pm$ 2.5	94.8$\pm$3	91.7 $\pm$ 2	80.1 $\pm$ 2.9	91.9 $\pm$ 4	87.9
RoB _{large} (Adpt ^H)†	6.0M	89.9 $\pm$ 5	96.2 $\pm$ 3	88.7 $\pm$ 2.9	66.5 $\pm$ 4.4	94.7 $\pm$ 2	92.1 $\pm$ 1	83.4 $\pm$ 1.1	91.0 $\pm$ 1.7	87.8
RoB _{large} (Adpt ^H)†	0.8M	90.3 $\pm$ 3	96.3 $\pm$ 5	87.7 $\pm$ 1.7	66.3 $\pm$ 2.0	94.7 $\pm$ 2	91.5 $\pm$ 1	72.9 $\pm$ 2.9	91.5 $\pm$ 5	86.4
RoB _{large} (LoRA)†	0.8M	90.6$\pm$2	96.2 $\pm$ 5	90.2$\pm$1.0	68.2 $\pm$ 1.9	94.8$\pm$3	91.6 $\pm$ 2	85.2$\pm$1.1	92.3$\pm$5	88.6
DeB _{XXL} (FT)*	1500.0M	91.8	97.2	92.0	72.0	96.0	92.7	93.9	92.9	91.1
DeB _{XXL} (LoRA)	4.7M	91.9$\pm$2	96.9 $\pm$ 2	92.6$\pm$6	72.4$\pm$1.1	96.0$\pm$1	92.9$\pm$1	94.9$\pm$4	93.0$\pm$2	91.3

Table 2: RoBERTa_{base}, RoBERTa_{large}, and DeBERTa_{XXL} with different adaptation methods on the GLUE benchmark. We report the overall (matched and mismatched) accuracy for MNLI, Matthew’s correlation for CoLA, Pearson correlation for STS-B, and accuracy for other tasks. Higher is better for all metrics. * indicates numbers published in prior works. † indicates runs configured in a setup similar to Houlsby et al. (2019) for a fair comparison.

Bias-only or BitFit is a baseline where we only train the bias vectors while freezing everything else. Contemporarily, this baseline has also been studied by BitFit (Zaken et al., 2021).

Prefix-embedding tuning (PreEmbed) inserts special tokens among the input tokens. These special tokens have trainable word embeddings and are generally not in the model’s vocabulary. Where to place such tokens can have an impact on performance. We focus on “prefixing”, which prepends such tokens to the prompt, and “infixing”, which appends to the prompt; both are discussed in Li & Liang (2021). We use l_p (resp. l_i) denote the number of prefix (resp. infix) tokens. The number of trainable parameters is $|\Theta| = d_{model} \times (l_p + l_i)$.

Prefix-layer tuning (PreLayer) is an extension to prefix-embedding tuning. Instead of just learning the word embeddings (or equivalently, the activations after the embedding layer) for some special tokens, we learn the activations after every Transformer layer. The activations computed from previous layers are simply replaced by trainable ones. The resulting number of trainable parameters is $|\Theta| = L \times d_{model} \times (l_p + l_i)$, where L is the number of Transformer layers.

Adapter tuning as proposed in Houlsby et al. (2019) inserts adapter layers between the self-attention module (and the MLP module) and the subsequent residual connection. There are two fully connected layers with biases in an adapter layer with a nonlinearity in between. We call this original design **Adapter^H**. Recently, Lin et al. (2020) proposed a more efficient design with the adapter layer applied only after the MLP module and after a LayerNorm. We call it **Adapter^L**. This is very similar to another design proposed in Pfeiffer et al. (2021), which we call **Adapter^P**. We also include another baseline call AdapterDrop (Rücklé et al., 2020) which drops some adapter layers for greater efficiency (**Adapter^D**). We cite numbers from prior works whenever possible to maximize the number of baselines we compare with; they are in rows with an asterisk (*) in the first column. In all cases, we have $|\Theta| = \hat{L}_{Adpt} \times (2 \times d_{model} \times r + r + d_{model}) + 2 \times \hat{L}_{LN} \times d_{model}$ where $\hat{L}_{Adpt}$ is the number of adapter layers and $\hat{L}_{LN}$ the number of trainable LayerNorms (e.g., in Adapter^L).

LoRA adds trainable pairs of rank decomposition matrices in parallel to existing weight matrices. As mentioned in Section 4.2, we only apply LoRA to W_q and W_v in most experiments for simplicity. The number of trainable parameters is determined by the rank r and the shape of the original weights: $|\Theta| = 2 \times \hat{L}_{LoRA} \times d_{model} \times r$, where $\hat{L}_{LoRA}$ is the number of weight matrices we apply LoRA to.

Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB _{base} (FT)*	125.0M	87.6	94.8	90.2	63.6	92.8	91.9	78.7	91.2	86.4
RoB _{base} (BitFit)*	0.1M	84.7	93.7	92.7	62.0	91.8	84.0	81.5	90.8	85.2
RoB _{base} (Adpt ^D)*	0.3M	87.1 $\pm$ 0	94.2 $\pm$ 1	88.5 $\pm$ 1.1	60.8 $\pm$ 4	93.1 $\pm$ 1	90.2 $\pm$ 0	71.5 $\pm$ 2.7	89.7 $\pm$ 3	84.4
RoB _{base} (Adpt ^D)*	0.9M	87.3 $\pm$ 1	94.7 $\pm$ 3	88.4 $\pm$ 1	62.6 $\pm$ 9	93.0 $\pm$ 2	90.6 $\pm$ 0	75.9 $\pm$ 2.2	90.3 $\pm$ 1	85.4
RoB _{base} (LoRA)	0.3M	87.5 $\pm$ 3	95.1$\pm$2	89.7 $\pm$ 7	63.4 $\pm$ 1.2	93.3$\pm$3	90.8 $\pm$ 1	86.6$\pm$7	91.5$\pm$2	87.2
RoB _{large} (FT)*	355.0M	90.2	96.4	90.9	68.0	94.7	92.2	86.6	92.4	88.9
RoB _{large} (LoRA)	0.8M	90.6$\pm$2	96.2 $\pm$ 5	90.9$\pm$1.2	68.2$\pm$1.9	94.9$\pm$3	91.6 $\pm$ 1	87.4$\pm$2.5	92.6$\pm$2	89.0
RoB _{large} (Adpt ^P)†	3.0M	90.2 $\pm$ 3	96.1 $\pm$ 3	90.2 $\pm$ 7	68.3$\pm$1.0	94.8$\pm$2	91.9$\pm$1	83.8 $\pm$ 2.9	92.1 $\pm$ 7	88.4
RoB _{large} (Adpt ^P)†	0.8M	90.5$\pm$3	96.6$\pm$2	89.7 $\pm$ 1.2	67.8 $\pm$ 2.5	94.8$\pm$3	91.7 $\pm$ 2	80.1 $\pm$ 2.9	91.9 $\pm$ 4	87.9
RoB _{large} (Adpt ^H)†	6.0M	89.9 $\pm$ 5	96.2 $\pm$ 3	88.7 $\pm$ 2.9	66.5 $\pm$ 4.4	94.7 $\pm$ 2	92.1 $\pm$ 1	83.4 $\pm$ 1.1	91.0 $\pm$ 1.7	87.8
RoB _{large} (Adpt ^H)†	0.8M	90.3 $\pm$ 3	96.3 $\pm$ 5	87.7 $\pm$ 1.7	66.3 $\pm$ 2.0	94.7 $\pm$ 2	91.5 $\pm$ 1	72.9 $\pm$ 2.9	91.5 $\pm$ 5	86.4
RoB _{large} (LoRA)†	0.8M	90.6$\pm$2	96.2 $\pm$ 5	90.2$\pm$1.0	68.2 $\pm$ 1.9	94.8$\pm$3	91.6 $\pm$ 2	85.2$\pm$1.1	92.3$\pm$5	88.6
DeB _{XXL} (FT)*	1500.0M	91.8	97.2	92.0	72.0	96.0	92.7	93.9	92.9	91.1
DeB _{XXL} (LoRA)	4.7M	91.9$\pm$2	96.9 $\pm$ 2	92.6$\pm$6	72.4$\pm$1.1	96.0$\pm$1	92.9$\pm$1	94.9$\pm$4	93.0$\pm$2	91.3

表2: RoBERTa_{base}、RoBERTa_{large} 和 DeBERTa_{XXL} 在 GLUE 基准上使用不同适应方法的表现。我们报告 MNLI 的总体（匹配和不匹配）准确率、CoLA 的 Matthew 相关系数、STS-B 的 Pearson 相关系数以及其他任务的准确率。所有指标中数值越高越好。* 表示前人工作的已发布数字。† 表示以类似 Houlsby 等（2019）的方法进行配置的运行，以便进行公平比较。

仅偏置或 BitFit 是一种基线方法，我们只训练偏置向量，同时冻结其他所有参数。在当代，这一基线也被 BitFit（Zaken 等，2021）研究过。

前缀嵌入调优（PreEmbed）在输入标记中插入特殊标记。这些特殊标记具有可训练的词嵌入，通常不在模型的词汇表中。放置这些标记的位置可能会影响性能。我们关注“前缀化”，即将这些标记添加到提示的前面，以及“中缀化”，即将其添加到提示的后面；两者都在 Li & Liang (2021) 中进行了讨论。我们使用 l_p （和 l_i ）分别表示前缀标记和中缀标记的数量。可训练参数的数量为 $|\Theta| = d_{model} \times (l_p + l_i)$ 。

前缀层调优（PreLayer）是对前缀嵌入调优的扩展。它不仅仅学习某些特殊标记的词嵌入（或等效地，嵌入层之后的激活值），而是学习每个 Transformer 层之后的激活值。前一层计算的激活值将被可训练的激活值替换。最终可训练参数的数量为 $|\Theta| = L \times d_{model} \times (l_p + l_i)$ ，其中 L 是 Transformer 层的数量。

正如 Houlsby 等人 (2019) 提出的，适配器调优是在自注意力模块（和 MLP 模块）与随后的残差连接之间插入适配器层。一个适配器层中有两个带偏置的全连接层，之间有一个非线性函数。我们称这种原始设计为 Adapter^H。最近，Lin 等人 (2020) 提出了一个更高效的设计，该设计仅在 MLP 模块之后以及 LayerNorm 之后应用适配器层。我们称其为 Adapter^L。这与 Pfeiffer 等人 (2021) 提出的另一个设计非常相似，我们称其为 Adapter^P。我们还包括了另一个基线称为 AdapterDrop（Rücklé 等人，2020），它通过丢弃一些适配器层以提高效率（Adapter^D）。我们尽可能引用先前工作的数据，以最大化我们对比的基线数量；它们在第一列带星号 (*) 的行中。在所有情况下，我们有 $|\Theta| = \hat{L}_{Adpt} \times (2 \times d_{model} \times r + r + d_{model}) + 2 \times \hat{L}_{LN} \times d_{model}$ ，其中 $\hat{L}_{Adpt}$ 是适配器层的数量， $\hat{L}_{LN}$ 是数量。

LoRA 在现有权重矩阵的基础上并行添加可训练的秩分解矩阵对。如第 4.2 节所述，为了简化起见，在大多数实验中我们仅将 LoRA 应用于 W_q 和 W_v 。可训练参数的数量由秩 r 以及原始权重的形状决定： $|\Theta| = 2 \times \hat{L}_{LoRA} \times d_{model} \times r$ ，其中 $\hat{L}_{LoRA}$ 是我们应用 LoRA 的权重矩阵数量。

Model & Method	# Trainable Parameters	E2E NLG Challenge				
		BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M (Adapter ^L)*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M (Adapter ^L)*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M (Adapter ^H)	11.09M	67.3 $\pm$.6	8.50 $\pm$.07	46.0 $\pm$.2	70.7 $\pm$.2	2.44 $\pm$.01
GPT-2 M (FT ^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	70.4$\pm$.1	8.85$\pm$.02	46.8$\pm$.2	71.8$\pm$.1	2.53$\pm$.02
GPT-2 L (FT)*	774.03M	68.5	8.78	46.0	69.9	2.45
GPT-2 L (Adapter ^L)	0.88M	69.1 $\pm$.1	8.68 $\pm$.03	46.3 $\pm$.0	71.4 $\pm$.2	2.49$\pm$.0
GPT-2 L (Adapter ^L)	23.00M	68.9 $\pm$.3	8.70 $\pm$.04	46.1 $\pm$.1	71.3 $\pm$.2	2.45 $\pm$.02
GPT-2 L (PreLayer)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	70.4$\pm$.1	8.89$\pm$.02	46.8$\pm$.2	72.0$\pm$.2	2.47 $\pm$.02

Table 3: GPT-2 medium (M) and large (L) with different adaptation methods on the E2E NLG Challenge. For all metrics, higher is better. LoRA outperforms several baselines with comparable or fewer trainable parameters. Confidence intervals are shown for experiments we ran. * indicates numbers published in prior works.

5.2 ROBERTA BASE/LARGE

RoBERTa (Liu et al., 2019) optimized the pre-training recipe originally proposed in BERT (Devlin et al., 2019a) and boosted the latter’s task performance without introducing many more trainable parameters. While RoBERTa has been overtaken by much larger models on NLP leaderboards such as the GLUE benchmark (Wang et al., 2019) in recent years, it remains a competitive and popular pre-trained model for its size among practitioners. We take the pre-trained RoBERTa base (125M) and RoBERTa large (355M) from the HuggingFace Transformers library (Wolf et al., 2020) and evaluate the performance of different efficient adaptation approaches on tasks from the GLUE benchmark. We also replicate Houlsby et al. (2019) and Pfeiffer et al. (2021) according to their setup. To ensure a fair comparison, we make two crucial changes to how we evaluate LoRA when comparing with adapters. First, we use the same batch size for all tasks and use a sequence length of 128 to match the adapter baselines. Second, we initialize the model to the pre-trained model for MRPC, RTE, and STS-B, not a model already adapted to MNLI like the fine-tuning baseline. Runs following this more restricted setup from Houlsby et al. (2019) are labeled with †. The result is presented in Table 2 (Top Three Sections). See Section D.1 for details on the hyperparameters used.

5.3 DeBERTa XXL

DeBERTa (He et al., 2021) is a more recent variant of BERT that is trained on a much larger scale and performs very competitively on benchmarks such as GLUE (Wang et al., 2019) and SuperGLUE (Wang et al., 2020). We evaluate if LoRA can still match the performance of a fully fine-tuned DeBERTa XXL (1.5B) on GLUE. The result is presented in Table 2 (Bottom Section). See Section D.2 for details on the hyperparameters used.

5.4 GPT-2 MEDIUM/LARGE

Having shown that LoRA can be a competitive alternative to full fine-tuning on NLU, we hope to answer if LoRA still prevails on NLG models, such as GPT-2 medium and large (Radford et al., b). We keep our setup as close as possible to Li & Liang (2021) for a direct comparison. Due to space constraint, we only present our result on E2E NLG Challenge (Table 3) in this section. See Section F.1 for results on WebNLG (Gardent et al., 2017) and DART (Nan et al., 2020). We include a list of the hyperparameters used in Section D.3.

Model & Method	# Trainable Parameters	E2E NLG Challenge				
		BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M (Adapter ^L)*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M (Adapter ^L)*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M (Adapter ^H)	11.09M	67.3 $\pm$.6	8.50 $\pm$.07	46.0 $\pm$.2	70.7 $\pm$.2	2.44 $\pm$.01
GPT-2 M (FT ^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	70.4$\pm$.1	8.85$\pm$.02	46.8$\pm$.2	71.8$\pm$.1	2.53$\pm$.02
GPT-2 L (FT)*	774.03M	68.5	8.78	46.0	69.9	2.45
GPT-2 L (Adapter ^L)	0.88M	69.1 $\pm$.1	8.68 $\pm$.03	46.3 $\pm$.0	71.4 $\pm$.2	2.49$\pm$.0
GPT-2 L (Adapter ^L)	23.00M	68.9 $\pm$.3	8.70 $\pm$.04	46.1 $\pm$.1	71.3 $\pm$.2	2.45 $\pm$.02
GPT-2 L (PreLayer)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	70.4$\pm$.1	8.89$\pm$.02	46.8$\pm$.2	72.0$\pm$.2	2.47 $\pm$.02

表 3: 在 E2E NLG 挑战中, 不同适应方法下的 GPT-2 中型 (M) 和大型 (L)。对于所有指标, 数值越高越好。LoRA 在可比或更少的可训练参数下优于若干基线。我们运行的实验显示了置信区间。* 表示前人工作中公布的数字。

5.2 ROBERTA 基础/大型

RoBERTa (Liu 等, 2019) 优化了最初由 BERT (Devlin 等, 2019a) 提出的预训练方法, 并在没有引入更多可训练参数的情况下提升了后者的任务性能。虽然近年来 RoBERTa 在诸如 GLUE 基准测试 (Wang 等, 2019) 等 NLP 排行榜上已被更大型的模型超越, 但对于其实效, 它仍然是一个具有竞争力且广受欢迎的预训练模型。我们从 HuggingFace Transformers 库 (Wolf 等, 2020) 获取预训练的 RoBERTa base (1.25 亿) 和 RoBERTa large (3.55 亿), 并评估不同高效适应方法在 GLUE 基准任务上的表现。我们还根据 Houlsby 等 (2019) 和 Pfeiffer 等 (2021) 的设置进行了复现。为了确保公平比较, 我们在将 LoRA 与适配器对比时对评估方式做了两个关键更改。首先, 我们对所有任务使用相同的批量大小, 并使用长度为 128 的序列以匹配适配器基线。其次, 我们

5.3 DEBERTA XXL

DeBERTa (He 等, 2021) 是 BERT 的一个较新的变体, 它在更大规模的数据上进行训练, 并在 GLUE (Wang 等, 2019) 和 SuperGLUE (Wang 等, 2020) 等基准测试中表现得非常具有竞争力。我们评估了 LoRA 是否仍然能够匹配在 GLUE 上完全微调的 DeBERTa XXL (1.5B) 的性能。结果见表 2 (底部部分)。有关使用的超参数的详细信息, 请参见第 D.2 节。

5.4 GPT-2 中型/大型

在展示了 LoRA 在 NLU 上可以成为全量微调的有竞争力的替代方案之后, 我们希望回答 LoRA 是否仍然在 NLG 模型上占优, 例如 GPT-2 中型和大型 (Radford 等, b)。我们尽可能保持设置与 Li & Liang (2021) 一致, 以便进行直接比较。由于篇幅限制, 本节仅展示我们在 E2E NLG 挑战赛上的结果 (表 3)。关于 WebNLG (Gardent 等, 2017) 和 DART (Nan 等, 2020) 的结果, 请参见附录 F.1。我们在附录 D.3 中列出了所使用的超参数清单。

Model&Method	# Trainable Parameters	WikiSQL	MNLI-m	SAMSum
		Acc. (%)	Acc. (%)	R1/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

Table 4: Performance of different adaptation methods on GPT-3 175B. We report the logical form validation accuracy on WikiSQL, validation accuracy on MultiNLI-matched, and Rouge-1/2/L on SAMSum. LoRA performs better than prior approaches, including full fine-tuning. The results on WikiSQL have a fluctuation around $\pm 0.5\%$, MNLI-m around $\pm 0.1\%$, and SAMSum around $\pm 0.2/\pm 0.2/\pm 0.1$ for the three metrics.

5.5 SCALING UP TO GPT-3 175B

As a final stress test for LoRA, we scale up to GPT-3 with 175 billion parameters. Due to the high training cost, we only report the typical standard deviation for a given task over random seeds, as opposed to providing one for every entry. See Section D.4 for details on the hyperparameters used.

As shown in Table 4, LoRA matches or exceeds the fine-tuning baseline on all three datasets. Note that not all methods benefit monotonically from having more trainable parameters, as shown in Figure 2. We observe a significant performance drop when we use more than 256 special tokens for prefix-embedding tuning or more than 32 special tokens for prefix-layer tuning. This corroborates similar observations in Li & Liang (2021). While a thorough investigation into this phenomenon is out-of-scope for this work, we suspect that having more special tokens causes the input distribution to shift further away from the pre-training data distribution. Separately, we investigate the performance of different adaptation approaches in the low-data regime in Section F.3.

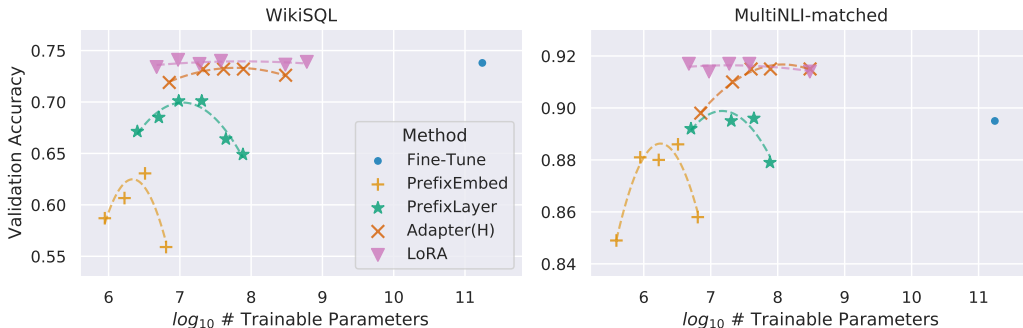


Figure 2: GPT-3 175B validation accuracy vs. number of trainable parameters of several adaptation methods on WikiSQL and MNLI-matched. LoRA exhibits better scalability and task performance. See Section F.2 for more details on the plotted data points.

6 RELATED WORKS

Transformer Language Models. Transformer (Vaswani et al., 2017) is a sequence-to-sequence architecture that makes heavy use of self-attention. Radford et al. (a) applied it to autoregressive language modeling by using a stack of Transformer decoders. Since then, Transformer-based language models have dominated NLP, achieving the state-of-the-art in many tasks. A new paradigm emerged with BERT (Devlin et al., 2019b) and GPT-2 (Radford et al., b) – both are large Transformer lan-

Model&Method	# Trainable Parameters	WikiSQL	MNLI-m	SAMSum
		Acc. (%)	Acc. (%)	R1/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

表 4: 不同适应方法在 GPT-3 175B 上的性能。我们报告了 WikiSQL 上的逻辑形式验证准确率、MultiNLI-matched 上的验证准确率, 以及 SAMSum 的 Rouge-1/2/L。LoRA 的表现优于先前的方法, 包括完全微调。在 WikiSQL 上的结果波动约为 $\pm 0.5\%$, MNLI-m 约为 $\pm 0.1\%$, SAMSum 三个指标分别约为 $\pm 0.2/\pm 0.2/\pm 0.1$ 。

5.5 扩展到 GPT-3 175B

作为对 LoRA 的最终压力测试, 我们将规模扩大到参数量为 1750 亿的 GPT-3。由于训练成本极高, 我们仅报告给定任务在随机种子下的典型标准差, 而不是为每个条目提供一个。有关使用的超参数的详细信息, 请参见 D.4 节。

如表4所示, LoRA在所有三个数据集上与微调基线相当或更优。请注意, 如图2所示, 并非所有方法随着可训练参数增多都会单调受益。当我们在前缀嵌入调优中使用超过256个特殊标记, 或在前缀层调优中使用超过32个特殊标记时, 性能会显著下降。这与Li & Liang (2021)中的类似观察结果相一致。虽然对这一现象进行彻底研究超出了本文的范围, 但我们怀疑更多的特殊标记会导致输入分布进一步偏离预训练数据分布。另请参见F.3节, 我们在低数据量情况下调查了不同适配方法的性能。

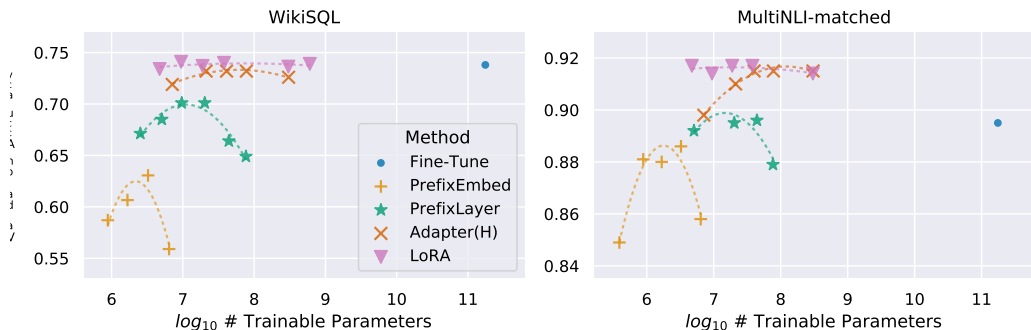


图 2: GPT-3 175B 在 WikiSQL 和 MNLI-matched 上使用多种适应方法时, 可训练参数数量与验证准确率的关系。LoRA 展现了更好的可扩展性和任务性能。绘制的数据点的更多细节请参见 F.2 节。

6 相关工作

变压器语言模型。变压器 (Vaswani 等, 2017) 是一种序列到序列的架构, 广泛使用自注意力机制。Radford 等 (a) 通过使用一系列变压器解码器将其应用于自回归语言建模。从那时起, 基于变压器的语言模型在自然语言处理领域占据主导地位, 在许多任务中实现了最先进的水平。一种新的范式随 BERT (Devlin 等, 2019b) 和 GPT-2 (Radford 等, b) 出现——两者都是大型变压器语言模型。

guage models trained on a large amount of text – where fine-tuning on task-specific data after pre-training on general domain data provides a significant performance gain compared to training on task-specific data directly. Training larger Transformers generally results in better performance and remains an active research direction. GPT-3 (Brown et al., 2020) is the largest single Transformer language model trained to-date with 175B parameters.

Prompt Engineering and Fine-Tuning. While GPT-3 175B can adapt its behavior with just a few additional training examples, the result depends heavily on the input prompt (Brown et al., 2020). This necessitates an empirical art of composing and formatting the prompt to maximize a model’s performance on a desired task, which is known as prompt engineering or prompt hacking. Fine-tuning retrains a model pre-trained on general domains to a specific task Devlin et al. (2019b); Radford et al. (a). Variants of it include learning just a subset of the parameters Devlin et al. (2019b); Collobert & Weston (2008), yet practitioners often retrain all of them to maximize the downstream performance. However, the enormity of GPT-3 175B makes it challenging to perform fine-tuning in the usual way due to the large checkpoint it produces and the high hardware barrier to entry since it has the same memory footprint as pre-training.

Parameter-Efficient Adaptation. Many have proposed inserting *adapter* layers between existing layers in a neural network (Houlsby et al., 2019; Rebuffi et al., 2017; Lin et al., 2020). Our method uses a similar bottleneck structure to impose a low-rank constraint on the weight updates. The key functional difference is that our learned weights can be merged with the main weights during inference, thus not introducing any latency, which is not the case for the adapter layers (Section 3). A contemporary extension of adapter is COMPACTER (Mahabadi et al., 2021), which essentially parametrizes the adapter layers using Kronecker products with some predetermined weight sharing scheme. Similarly, combining LoRA with other tensor product-based methods could potentially improve its parameter efficiency, which we leave to future work. More recently, many proposed optimizing the input word embeddings in lieu of fine-tuning, akin to a continuous and differentiable generalization of prompt engineering (Li & Liang, 2021; Lester et al., 2021; Hambardzumyan et al., 2020; Liu et al., 2021). We include comparisons with Li & Liang (2021) in our experiment section. However, this line of works can only scale up by using more special tokens in the prompt, which take up available sequence length for task tokens when positional embeddings are learned.

Low-Rank Structures in Deep Learning. Low-rank structure is very common in machine learning. A lot of machine learning problems have certain intrinsic low-rank structure (Li et al., 2016; Cai et al., 2010; Li et al., 2018b; Grasedyck et al., 2013). Moreover, it is known that for many deep learning tasks, especially those with a heavily over-parametrized neural network, the learned neural network will enjoy low-rank properties after training (Oymak et al., 2019). Some prior works even explicitly impose the low-rank constraint when training the original neural network (Sainath et al., 2013; Povey et al., 2018; Zhang et al., 2014; Jaderberg et al., 2014; Zhao et al., 2016; Khodak et al., 2021; Denil et al., 2014); however, to the best of our knowledge, none of these works considers low-rank update to a frozen model for *adaptation to downstream tasks*. In theory literature, it is known that neural networks outperform other classical learning methods, including the corresponding (finite-width) neural tangent kernels (Allen-Zhu et al., 2019; Li & Liang, 2018) when the underlying concept class has certain low-rank structure (Ghorbani et al., 2020; Allen-Zhu & Li, 2019; Allen-Zhu & Li, 2020a). Another theoretical result in Allen-Zhu & Li (2020b) suggests that low-rank adaptations can be useful for adversarial training. In sum, we believe that our proposed low-rank adaptation update is well-motivated by the literature.

7 UNDERSTANDING THE LOW-RANK UPDATES

Given the empirical advantage of LoRA, we hope to further explain the properties of the low-rank adaptation learned from downstream tasks. Note that the low-rank structure not only lowers the hardware barrier to entry which allows us to run multiple experiments in parallel, but also gives better interpretability of how the update weights are correlated with the pre-trained weights. We focus our study on GPT-3 175B, where we achieved the largest reduction of trainable parameters (up to $10,000\times$) without adversely affecting task performances.

We perform a sequence of empirical studies to answer the following questions: 1) Given a parameter budget constraint, *which subset of weight matrices* in a pre-trained Transformer should we adapt

在大量文本上训练的语言模型——在对通用领域数据进行预训练后，再对特定任务数据进行微调，相较于直接在特定任务数据上训练，可以显著提升性能。训练更大的Transformer模型通常会带来更好的性能，并且仍然是一个活跃的研究方向。GPT-3 (Brown 等, 2020) 是迄今为止训练的最大的单一Transformer语言模型，拥有1750亿个参数。

提示工程与微调。尽管 GPT-3 175B 只需少量额外的训练示例即可调整其行为，但结果在很大程度上取决于输入提示 (Brown 等, 2020)。这需要一种经验性的艺术，即编写和格式化提示以最大化模型在所需任务上的性能，这被称为提示工程或提示黑客。微调则是将一个在通用领域预训练的模型重新训练以适应特定任务 (Devlin 等, 2019b; Radford 等, a)。其变体包括仅学习部分参数 (Devlin 等, 2019b; Collobert & Weston, 2008)，但实践者通常会重新训练所有参数以最大化下游性能。然而，由于 GPT-3 175B 的庞大体量，使得以常规方式进行微调变得具有挑战性，因为它产生的大型检查点以及由于其与预训练相同的内存占用导致的高硬件门槛。

参数高效适配。许多人提出在神经网络的现有层之间插入`adapter`层 (Houlsby 等, 2019; Rebuffi 等, 2017; Lin 等, 2020)。我们的方法使用类似的瓶颈结构，对权重更新施加低秩约束。关键的功能性差异在于，我们学习到的权重可以在推理过程中与主权重合并，因此不会引入任何延迟，而适配器层则不具备这种特性 (第3节)。适配器的一个当代表扩展是 COMPACTER (Mahabadi 等, 2021)，其本质上是使用某种预定的权重共享方案通过 Kronecker 积来参数化适配器层。同样，将 LoRA 与其他基于张量积的方法结合，有可能提高其参数效率，这部分工作我们留待未来研究。最近，许多人提出优化输入词嵌入来代替微调，这类似于提示工程的连续且可微分的一般化 (Li &

深度学习中的低秩结构。低秩结构在机器学习中非常常见。许多机器学习问题具有某种内在的低秩结构 (Li 等, 2016; Cai 等, 2010; Li 等, 2018b; Grasedyck 等, 2013)。此外，已知对于许多深度学习任务，尤其是那些高度过参数化的神经网络，训练后的神经网络会呈现低秩特性 (Oymak 等, 2019)。一些先前的工作甚至在训练原始神经网络时明确施加了低秩约束 (Sainath 等, 2013; Povey 等, 2018; Zhang 等, 2014; Jaderberg 等, 2014; Zhao 等, 2016; Khodak 等, 2021; Denil 等, 2014)；然而，据我们所知，这些工作中没有考虑对冻结模型进行低秩更新。在理论文献中，已知神经网络的性能优于其他经典学习方法，包括相应的 (有限宽度) 神经切线核

7 理解低秩更新

鉴于 LoRA 的经验优势，我们希望进一步解释从下游任务中学习到的低秩适应的特性。请注意，低秩结构不仅降低了硬件门槛，使我们能够并行运行多个实验，还能更好地解释更新权重与预训练权重的关联方式。我们的研究重点是 GPT-3 175B，在该模型上我们实现了可训练参数的最大减少 (高达 10,000 $\times$)，而不会对任务性能产生不利影响。

我们进行一系列实证研究来回答以下问题：1) 在参数预算限制下，预训练 Transformer 中的 *which subset of weight matrices* 我们是否应该进行调整

to maximize downstream performance? 2) Is the “optimal” adaptation matrix ΔW *really rank-deficient*? If so, what is a good rank to use in practice? 3) What is the connection between ΔW and W ? Does ΔW highly correlate with W ? How large is ΔW comparing to W ?

We believe that our answers to question (2) and (3) shed light on the fundamental principles of using pre-trained language models for downstream tasks, which is a critical topic in NLP.

7.1 WHICH WEIGHT MATRICES IN TRANSFORMER SHOULD WE APPLY LoRA TO?

Given a limited parameter budget, which types of weights should we adapt with LoRA to obtain the best performance on downstream tasks? As mentioned in Section 4.2, we only consider weight matrices in the self-attention module. We set a parameter budget of 18M (roughly 35MB if stored in FP16) on GPT-3 175B, which corresponds to $r = 8$ if we adapt one type of attention weights or $r = 4$ if we adapt two types, for all 96 layers. The result is presented in Table 5.

	# of Trainable Parameters = 18M						
Weight Type	W_q	W_k	W_v	W_o	W_q, W_k	W_q, W_v	W_q, W_k, W_v, W_o
Rank r	8	8	8	8	4	4	2
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	73.7	73.7
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	91.7

Table 5: Validation accuracy on WikiSQL and MultiNLI after applying LoRA to different types of attention weights in GPT-3, given the same number of trainable parameters. Adapting both W_q and W_v gives the best performance overall. We find the standard deviation across random seeds to be consistent for a given dataset, which we report in the first column.

Note that putting all the parameters in ΔW_q or ΔW_k results in significantly lower performance, while adapting both W_q and W_v yields the best result. This suggests that even a rank of four captures enough information in ΔW such that it is preferable to adapt more weight matrices than adapting a single type of weights with a larger rank.

7.2 WHAT IS THE OPTIMAL RANK r FOR LoRA?

We turn our attention to the effect of rank r on model performance. We adapt $\{W_q, W_v\}$, $\{W_q, W_k, W_v, W_o\}$, and just W_q for a comparison.

	Weight Type	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL ($\pm 0.5\%$)	W_q	68.8	69.6	70.5	70.4	70.0
	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
MultiNLI ($\pm 0.1\%$)	W_q	90.7	90.9	91.1	90.7	90.7
	W_q, W_v	91.3	91.4	91.3	91.6	91.4
	W_q, W_k, W_v, W_o	91.2	91.7	91.7	91.5	91.4

Table 6: Validation accuracy on WikiSQL and MultiNLI with different rank r . To our surprise, a rank as small as one suffices for adapting both W_q and W_v on these datasets while training W_q alone needs a larger r . We conduct a similar experiment on GPT-2 in Section H.2.

Table 6 shows that, surprisingly, LoRA already performs competitively with a very small r (more so for $\{W_q, W_v\}$ than just W_q). This suggests the update matrix ΔW could have a very small “intrinsic rank”.⁶ To further support this finding, we check the overlap of the subspaces learned by different choices of r and by different random seeds. We argue that increasing r does not cover a more meaningful subspace, which suggests that a low-rank adaptation matrix is sufficient.

⁶However, we do not expect a small r to work for every task or dataset. Consider the following thought experiment: if the downstream task were in a different language than the one used for pre-training, retraining the entire model (similar to LoRA with $r = d_{model}$) could certainly outperform LoRA with a small r .

为了最大化下游性能？2) “最优”的适应矩阵是 ΔW *really rank-deficient* 吗？如果是，实际中使用的一个好的秩是多少？3) ΔW 和 W 之间有什么联系？ ΔW 与 W 高度相关吗？相比于 W ， ΔW 有多大？

我们相信，我们对问题（2）和（3）的回答阐明了在下游任务中使用预训练语言模型的基本原则，这在自然语言处理（NLP）中是一个关键话题。

7.1 我们应该将 LoRA 应用于 Transformer 的哪些权重矩阵？

在有限的参数预算下，我们应该调整哪种类型的权重以在下游任务中获得最佳性能？如第4.2节所述，我们只考虑自注意力模块中的权重矩阵。我们在GPT-3 175B上设置了18M的参数预算（如果以FP16存储大约为35MB），如果我们只调整一种注意力权重，则对应 $r = 8$ ，如果我们调整两种类型，则对应 $r = 4$ ，适用于所有96层。结果如表5所示。

	# of Trainable Parameters = 18M						
Weight Type Rank r	W_q 8	W_k 8	W_v 8	W_o 8	W_q, W_k 4	W_q, W_v 4	W_q, W_k, W_v, W_o 2
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	73.7	73.7
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	91.7

表5：在GPT-3中对不同类型的注意力权重应用LoRA后，在WikiSQL和MultiNLI上的验证精度，在训练参数数量相同的情况下。同时适配 W_q 和 W_v 整体上表现最佳。我们发现，对于给定数据集，不同随机种子的标准差是一致的，我们在第一列中报告了这一点。

请注意，将所有参数放入 ΔW_q 或 ΔW_k 会导致性能显著下降，而同时调整 W_q 和 W_v 则能获得最佳结果。这表明，即使是四阶矩阵也能在 ΔW 中捕捉到足够的信息，因此相比于使用更大阶数调整单一类型的权重，调整更多的权重矩阵更为优选。

7.2 LORA 的最佳秩 r 是什么？

我们将注意力转向等级 r 对模型性能的影响。我们调整了 $\{W_q, W_v\}$ 、 $\{W_q, W_k, W_v, W_o\}$ ，以及仅 W_q 进行比较。

	Weight Type	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL ($\pm 0.5\%$)	W_q	68.8	69.6	70.5	70.4	70.0
	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
MultiNLI ($\pm 0.1\%$)	W_q	90.7	90.9	91.1	90.7	90.7
	W_q, W_v	91.3	91.4	91.3	91.6	91.4
	W_q, W_k, W_v, W_o	91.2	91.7	91.7	91.5	91.4

表6：在WikiSQL和MultiNLI上使用不同秩的验证准确率 r 。令人惊讶的是，即使秩小到1，也足以在这些数据集上适配 W_q 和 W_v ，而单独训练 W_q 则需要更大的 r 。我们在第H.2节对GPT-2进行了类似的实验。

表6显示，令人惊讶的是，LoRA 即使在一个非常小的 r （上也能表现得很有竞争力，对 $\{W_q, W_v\}$ 的效果甚至比 W_q ）更明显。这表明更新矩阵 ΔW 可能具有非常小的“内在秩”。⁶为了进一步支持这一发现，我们检查了由不同 r 选择和不同随机种子学习到的子空间的重叠情况。我们认为，增加 r 并不会覆盖一个更有意义的子空间，这表明一个低秩的适配矩阵是足够的。

⁶然而，我们并不期望一个小的 r 能够适用于每一个任务或数据集。考虑以下思想实验：如果下游任务使用的语言与预训练所用的语言不同，那么对整个模型进行再训练（类似于使用 $r = d_{model}$ 的 LoRA）肯定可以超越使用一个小 r 的 LoRA。

Subspace similarity between different r . Given $A_{r=8}$ and $A_{r=64}$ which are the learned adaptation matrices with rank $r = 8$ and 64 using the *same pre-trained model*, we perform singular value decomposition and obtain the right-singular unitary matrices $U_{A_{r=8}}$ and $U_{A_{r=64}}$.⁷ We hope to answer: how much of the subspace spanned by the top i singular vectors in $U_{A_{r=8}}$ (for $1 \leq i \leq 8$) is contained in the subspace spanned by top j singular vectors of $U_{A_{r=64}}$ (for $1 \leq j \leq 64$)? We measure this quantity with a normalized subspace similarity based on the Grassmann distance (See Appendix G for a more formal discussion)

$$\phi(A_{r=8}, A_{r=64}, i, j) = \frac{\|U_{A_{r=8}}^{i\top} U_{A_{r=64}}^j\|_F^2}{\min(i, j)} \in [0, 1] \quad (4)$$

where $U_{A_{r=8}}^i$ represents the columns of $U_{A_{r=8}}$ corresponding to the top- i singular vectors.

$\phi(\cdot)$ has a range of $[0, 1]$, where 1 represents a complete overlap of subspaces and 0 a complete separation. See Figure 3 for how ϕ changes as we vary i and j . We only look at the 48th layer (out of 96) due to space constraint, but the conclusion holds for other layers as well, as shown in Section H.1.

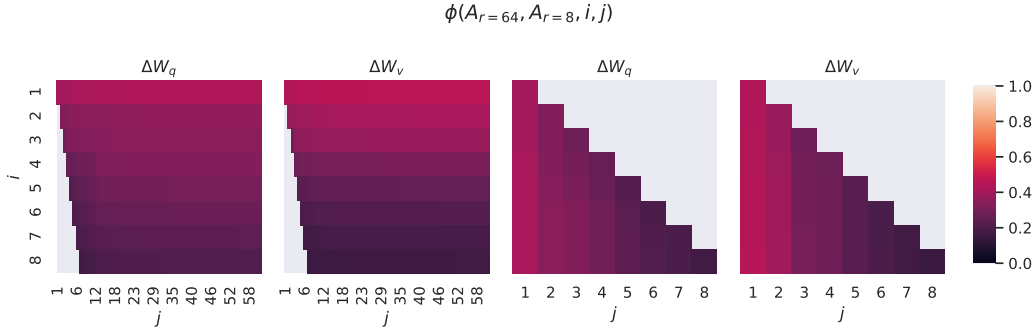


Figure 3: Subspace similarity between column vectors of $A_{r=8}$ and $A_{r=64}$ for both ΔW_q and ΔW_v . The third and the fourth figures zoom in on the lower-left triangle in the first two figures. The top directions in $r = 8$ are included in $r = 64$, and vice versa.

We make an *important observation* from Figure 3.

Directions corresponding to the top singular vector overlap significantly between $A_{r=8}$ and $A_{r=64}$, while others do not. Specifically, ΔW_v (resp. ΔW_q) of $A_{r=8}$ and ΔW_v (resp. ΔW_q) of $A_{r=64}$ share a subspace of dimension 1 with normalized similarity > 0.5 , providing an explanation of why $r = 1$ performs quite well in our downstream tasks for GPT-3.

Since both $A_{r=8}$ and $A_{r=64}$ are learned using the same pre-trained model, Figure 3 indicates that the top singular-vector directions of $A_{r=8}$ and $A_{r=64}$ are the most useful, while other directions potentially contain mostly random noises accumulated during training. Hence, the adaptation matrix can indeed have a very low rank.

Subspace similarity between different random seeds. We further confirm this by plotting the normalized subspace similarity between two randomly seeded runs with $r = 64$, shown in Figure 4. ΔW_q appears to have a higher “intrinsic rank” than ΔW_v , since more common singular value directions are learned by both runs for ΔW_q , which is in line with our empirical observation in Table 6. As a comparison, we also plot two random Gaussian matrices, which do not share any common singular value directions with each other.

7.3 HOW DOES THE ADAPTATION MATRIX ΔW COMPARE TO W ?

We further investigate the relationship between ΔW and W . In particular, does ΔW highly correlate with W ? (Or mathematically, is ΔW mostly contained in the top singular directions of W ?) Also,

⁷Note that a similar analysis can be carried out with B and the left-singular unitary matrices – we stick with A for our experiments.

不同 r 之间的子空间相似性。给定 $A_{r=8}$ 和 $A_{r=64}$ ，它们是使用 *same pre-trained model* 学习的秩为 $r =$ (分别为 8 和 64) 的适应矩阵，我们进行奇异值分解并得到右奇异酉矩阵 $U_{A_{r=8}}$ 和 $U_{A_{r=64}}$ 。⁷ 我们希望回答： $U_{A_{r=8}}$ (中前 i 个奇异向量所张成的子空间有多少包含在 $U_{A_{r=64}}$ (中前 j 个奇异向量所张成的子空间中，用于 $1 \leq i \leq 8$) 和 $1 \leq j \leq 64$)？我们使用基于 Grassmann 距离的归一化子空间相似性来测量该量（更正式的讨论见附录 G）。

$$\phi(A_{r=8}, A_{r=64}, i, j) = \frac{\|U_{A_{r=8}}^{i\top} U_{A_{r=64}}^j\|_F^2}{\min(i, j)} \in [0, 1] \quad (4)$$

其中 $U_{A_{r=8}}^i$ 表示 $U_{A_{r=8}}$ 的列，这些列对应于前 i 个奇异向量。

$\phi(\cdot)$ 其范围是 $[0, 1]$ ，其中 1 表示子空间完全重叠，0 表示完全分离。见图 3，了解当我们改变 i 和 j 时 ϕ 的变化。由于篇幅限制，我们只观察第 48 层（共 96 层），但如 H.1 节所示，结论对其他层也同样适用。

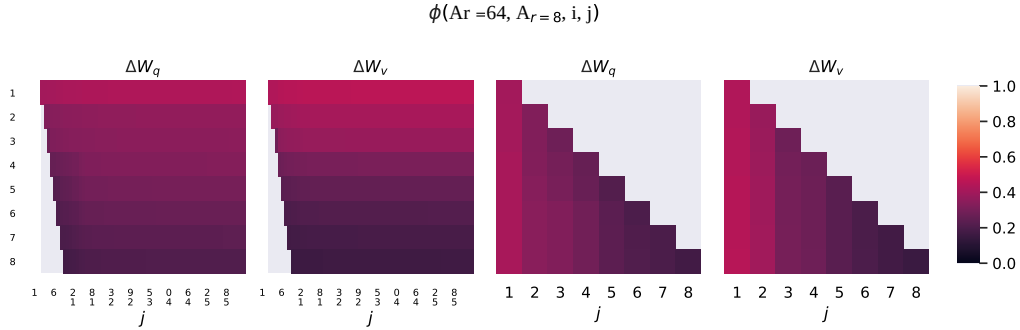


图 3: ΔW_q 和 ΔW_v 的 $A_{r=8}$ 与 $A_{r=64}$ 列向量之间的子空间相似性。第三和第四个图放大了前两个图的左下三角部分。 $r = 8$ 中的顶部方向包含在 $r = 64$ 中，反之亦然。

我们从图3制作一个 *important observation*。

与顶部奇异向量对应的方向在 $A_{r=8}$ 和 $A_{r=64}$ 之间有显著重叠，而其他方向则没有。具体来说， $A_{r=8}$ 的 ΔW_v (分别与 $A_{r=64}$ 的 ΔW_v (分别共享一个维度为 1 的子空间，归一化相似度为 > 0.5 ，这解释了为什么 $r = 1$ 在我们针对 GPT-3 的下游任务中表现相当好。

由于 $A_{r=8}$ 和 $A_{r=64}$ 都是使用相同的预训练模型学习的，图 3 表明 $A_{r=8}$ 和 $A_{r=64}$ 的主奇异向量方向是最有用的，而其他方向可能主要包含训练过程中积累的大部分随机噪声。因此，适配矩阵确实可以具有非常低的秩。

不同随机种子之间的子空间相似性。我们进一步通过绘制两个随机种子运行的归一化子空间相似性来确认这一点，如图4所示， $r = 64$ 。 ΔW_q 似乎比 ΔW_v 具有更高的“内在秩”，因为两个运行在 ΔW_q 上学到的奇异值方向更多是共同的，这与我们在表6中的经验观察一致。作为对比，我们还绘制了两个随机高斯矩阵，它们彼此之间不共享任何奇异值方向。

7.3 适应矩阵 ΔW 与 W 相比如何？

我们进一步研究 ΔW 和 W 之间的关系。具体来说， ΔW 是否与 W 高度相关？（或者数学上， ΔW 是否主要包含在 W 的前奇异方向中？）此外，

⁷Note that a similar analysis can be carried out with B and the left-singular unitary matrices – we stick with A for our experiments.

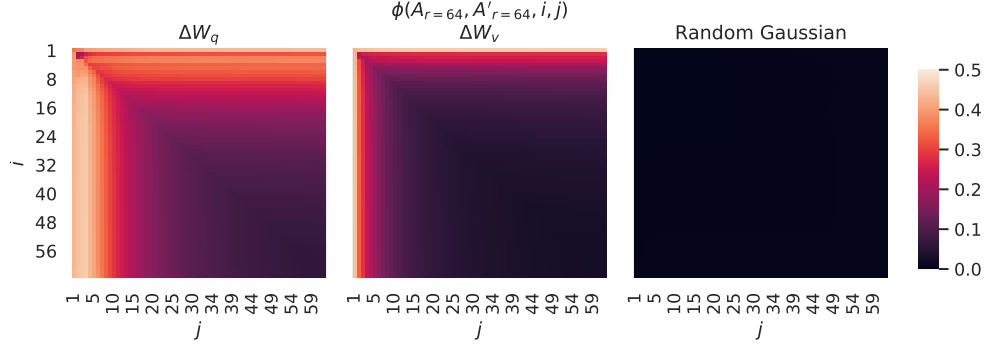


Figure 4: **Left and Middle:** Normalized subspace similarity between the column vectors of $A_{r=64}$ from two random seeds, for both ΔW_q and ΔW_v in the 48-th layer. **Right:** the same heat-map between the column vectors of two random Gaussian matrices. See Section H.1 for other layers.

how “large” is ΔW comparing to its corresponding directions in W ? This can shed light on the underlying mechanism for adapting pre-trained language models.

To answer these questions, we project W onto the r -dimensional subspace of ΔW by computing $U^\top W V^\top$, with U/V being the left/right singular-vector matrix of ΔW . Then, we compare the Frobenius norm between $\|U^\top W V^\top\|_F$ and $\|W\|_F$. As a comparison, we also compute $\|U^\top W V^\top\|_F$ by replacing U, V with the top r singular vectors of W or a random matrix.

	ΔW_q	$r = 4$ W_q	Random	ΔW_q	$r = 64$ W_q	Random
$\ U^\top W_q V^\top\ _F =$	0.32	21.67	0.02	1.90	37.71	0.33
$\ W_q\ _F = 61.95$	$\ \Delta W_q\ _F = 6.91$			$\ \Delta W_q\ _F = 3.57$		

Table 7: The Frobenius norm of $U^\top W_q V^\top$ where U and V are the left/right top r singular vector directions of either (1) ΔW_q , (2) W_q , or (3) a random matrix. The weight matrices are taken from the 48th layer of GPT-3.

We draw *several conclusions* from Table 7. First, ΔW has a stronger correlation with W compared to a random matrix, indicating that ΔW amplifies some features that are already in W . Second, instead of repeating the top singular directions of W , ΔW only *amplifies directions that are not emphasized in W* . Third, the amplification factor is rather huge: $21.5 \approx 6.91/0.32$ for $r = 4$. See Section H.4 for why $r = 64$ has a smaller amplification factor. We also provide a visualization in Section H.3 for how the correlation changes as we include more top singular directions from W_q . This suggests that the low-rank adaptation matrix potentially *amplifies the important features for specific downstream tasks that were learned but not emphasized in the general pre-training model*.

8 CONCLUSION AND FUTURE WORK

Fine-tuning enormous language models is prohibitively expensive in terms of the hardware required and the storage/switching cost for hosting independent instances for different tasks. We propose LoRA, an efficient adaptation strategy that neither introduces inference latency nor reduces input sequence length while retaining high model quality. Importantly, it allows for quick task-switching when deployed as a service by sharing the vast majority of the model parameters. While we focused on Transformer language models, the proposed principles are generally applicable to any neural networks with dense layers.

There are many directions for future works. 1) LoRA can be combined with other efficient adaptation methods, potentially providing orthogonal improvement. 2) The mechanism behind fine-tuning or LoRA is far from clear – how are features learned during pre-training transformed to do well on downstream tasks? We believe that LoRA makes it more tractable to answer this than full fine-

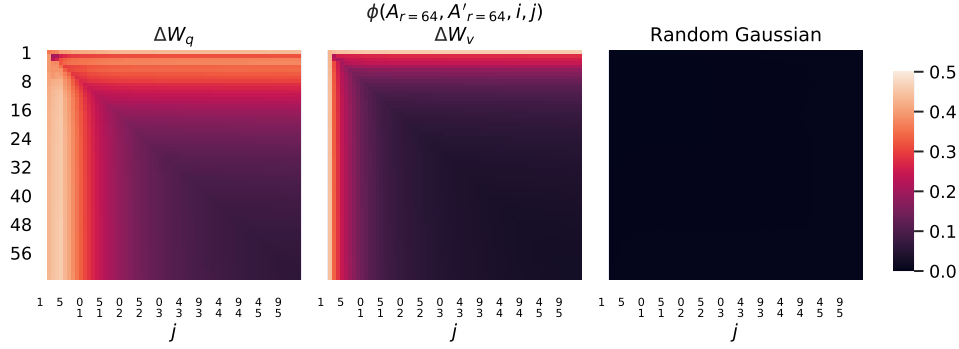


图4: 左和中: 48层中, 对于 ΔW_q 和 ΔW_v , 来自两个随机种子的 $A_{r=64}$ 列向量之间的归一化子空间相似性。右: 两个随机高斯矩阵的列向量之间的相同热图。其他层详见H.1节。

与 W 中相应方向相比, ΔW 有多“大”? 这可以揭示预训练语言模型适应的潜在机制。

为了回答这些问题, 我们通过计算 $U^T W V^T$ 将 W 投影到 ΔW 的 r 维子空间, 其中 U/V 是 ΔW 的左/右奇异向量矩阵。然后, 我们比较 $\|U^T W V^T\|_F$ 和 $\|W\|_F$ 之间的 Frobenius 范数。作为对比, 我们还通过将 U, V 替换为 W 的前 r 个奇异向量或随机矩阵来计算 $\|U^T W V^T\|_F$ 。

	$r = 4$			$r = 64$		
	ΔW_q	W_q	Random	ΔW_q	W_q	Random
$\ U^T W_q V^T\ _F =$	0.32	21.67	0.02	1.90	37.71	0.33
$\ W_q\ _F = 61.95$	$\ \Delta W_q\ _F = 6.91$			$\ \Delta W_q\ _F = 3.57$		

表 7: $U^T W_q V^T$ 的 Frobenius 范数, 其中 U 和 V 分别是 (1) ΔW_q , (2) W_q 或 (3) 一个随机矩阵的左/右前 r 奇异向量方向。权重矩阵取自 GPT-3 的第 48 层。

我们从表7中提取 *several conclusions*。首先, ΔW 与 W 的相关性比随机矩阵更强, 这表明 ΔW 放大了 W 中已经存在的一些特征。其次, ΔW 不仅仅重复 W 的主要奇异方向, 而是 *amplifies directions that are not emphasized in W* 。第三, 放大因子相当大: $21.5 \approx 6.91/0.32$ 对于 $r = 4$ 。关于为什么 $r = 64$ 的放大因子较小, 请参见 H.4 节。我们还在 H.3 节提供了可视化, 展示随着包含来自 W_q 的更多主要奇异方向相关性如何变化。这表明低秩适应矩阵可能 *amplifies the important features for specific downstream tasks that were learned but not emphasized in the general pre-training model*。

8 结论与未来工作

对巨型语言模型进行微调在所需硬件和为不同任务托管独立实例所需的存储/切换成本方面都是极其昂贵的。我们提出了 LoRA, 一种高效的适应策略, 它既不会引入推理延迟, 也不会缩短输入序列长度, 同时仍能保持高模型质量。重要的是, 它通过共享绝大多数模型参数, 使得在作为服务部署时可以快速切换任务。虽然我们关注的是 Transformer 语言模型, 但所提出的原则通常适用于任何具有密集层的神经网络。

未来的工作有许多方向。1) LoRA 可以与其他高效的适应方法结合, 可能提供正交的改进。2) 微调或 LoRA 背后的机制还远不清楚——在预训练过程中学习的特征是如何被转换以在下游任务中表现良好的? 我们相信, 与完全微调相比, LoRA 使得回答这个问题更容易。

tuning. 3) We mostly depend on heuristics to select the weight matrices to apply LoRA to. Are there more principled ways to do it? 4) Finally, the rank-deficiency of ΔW suggests that W could be rank-deficient as well, which can also be a source of inspiration for future works.

REFERENCES

- Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning. *arXiv:2012.13255 [cs]*, December 2020. URL <http://arxiv.org/abs/2012.13255>.
- Zeyuan Allen-Zhu and Yuanzhi Li. What Can ResNet Learn Efficiently, Going Beyond Kernels? In *NeurIPS*, 2019. Full version available at <http://arxiv.org/abs/1905.10337>.
- Zeyuan Allen-Zhu and Yuanzhi Li. Backward feature correction: How deep learning performs deep learning. *arXiv preprint arXiv:2001.04413*, 2020a.
- Zeyuan Allen-Zhu and Yuanzhi Li. Feature purification: How adversarial training performs robust deep learning. *arXiv preprint arXiv:2005.10190*, 2020b.
- Zeyuan Allen-Zhu, Yuanzhi Li, and Zhao Song. A convergence theory for deep learning via over-parameterization. In *ICML*, 2019. Full version available at <http://arxiv.org/abs/1811.03962>.
- Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language Models are Few-Shot Learners. *arXiv:2005.14165 [cs]*, July 2020. URL <http://arxiv.org/abs/2005.14165>.
- Jian-Feng Cai, Emmanuel J Candès, and Zuowei Shen. A singular value thresholding algorithm for matrix completion. *SIAM Journal on optimization*, 20(4):1956–1982, 2010.
- Daniel Cer, Mona Diab, Eneko Agirre, Inigo Lopez-Gazpio, and Lucia Specia. Semeval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation. *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, 2017. doi: 10.18653/v1/s17-2001. URL <http://dx.doi.org/10.18653/v1/S17-2001>.
- Ronan Collobert and Jason Weston. A unified architecture for natural language processing: deep neural networks with multitask learning. In *Proceedings of the 25th international conference on Machine learning, ICML '08*, pp. 160–167, New York, NY, USA, July 2008. Association for Computing Machinery. ISBN 978-1-60558-205-4. doi: 10.1145/1390156.1390177. URL <https://doi.org/10.1145/1390156.1390177>.
- Misha Denil, Babak Shakibi, Laurent Dinh, Marc’Aurelio Ranzato, and Nando de Freitas. Predicting parameters in deep learning, 2014.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019a.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv:1810.04805 [cs]*, May 2019b. URL <http://arxiv.org/abs/1810.04805>. arXiv: 1810.04805.
- William B. Dolan and Chris Brockett. Automatically constructing a corpus of sentential paraphrases. In *Proceedings of the Third International Workshop on Paraphrasing (IWP2005)*, 2005. URL <https://aclanthology.org/I05-5002>.
- Claire Gardent, Anastasia Shimorina, Shashi Narayan, and Laura Perez-Beltrachini. The webnlg challenge: Generating text from rdf data. In *Proceedings of the 10th International Conference on Natural Language Generation*, pp. 124–133, 2017.

调优。3) 我们主要依赖启发式方法来选择应用 LoRA 的权重矩阵。是否有更系统的方法来做这件事? 4) 最后, ΔW 的秩亏暗示 W 也可能是秩亏的, 这也可以成为未来工作的灵感来源。

参考文献

Armen Aghajanyan, Luke Zettlemoyer 和 Sonal Gupta。《内在维度解释了语言模型微调的有效性》。 *arXiv:2012.13255 [cs]*, 2020年12月。网址 <http://arxiv.org/abs/2012.13255>。Zeyuan Allen-Zhu 和 Yuanzhi Li。《ResNet 能高效学习什么, 超越核方法?》在 *NeurIPS*, 2019。完整版可在 <http://arxiv.org/abs/1905.10337> 获取。Zeyuan Allen-Zhu 和 Yuanzhi Li。《反向特征校正: 深度学习如何执行深度学习》。 *arXiv preprint arXiv:2001.04413*, 2020a。Zeyuan Allen-Zhu 和 Yuanzhi Li。《特征净化: 对抗训练如何实现稳健的深度学习》。 *arXiv preprint arXiv:2005.10190*, 2020b。Zeyuan Allen-Zhu, Yuanzhi Li, 和 Zhao Song。《通过参数化的深度学习收敛理论》。在 *ICML*, 2019。完整版可在 <http://arxiv.org/abs/1811.03962> 获取。Jimmy Lei Ba, Jamie Ryan Kiros, 和 Geoffrey E. Hinton。《层归一化》, 2016。Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish

-
- Behrooz Ghorbani, Song Mei, Theodor Misiakiewicz, and Andrea Montanari. When do neural networks outperform kernel methods? *arXiv preprint arXiv:2006.13409*, 2020.
- Bogdan Gliwa, Iwona Mochol, Maciej Biesek, and Aleksander Wawer. Samsun corpus: A human-annotated dialogue dataset for abstractive summarization. *CoRR*, abs/1911.12237, 2019. URL <http://arxiv.org/abs/1911.12237>.
- Lars Grasedyck, Daniel Kressner, and Christine Tobler. A literature survey of low-rank tensor approximation techniques. *GAMM-Mitteilungen*, 36(1):53–78, 2013.
- Jihun Ham and Daniel D. Lee. Grassmann discriminant analysis: a unifying view on subspace-based learning. In *ICML*, pp. 376–383, 2008. URL <https://doi.org/10.1145/1390156.1390204>.
- Karen Hambardzumyan, Hrant Khachatrian, and Jonathan May. WARP: Word-level Adversarial ReProgramming. *arXiv:2101.00121 [cs]*, December 2020. URL <http://arxiv.org/abs/2101.00121>. arXiv: 2101.00121.
- Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced bert with disentangled attention, 2021.
- Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-Efficient Transfer Learning for NLP. *arXiv:1902.00751 [cs, stat]*, June 2019. URL <http://arxiv.org/abs/1902.00751>.
- Max Jaderberg, Andrea Vedaldi, and Andrew Zisserman. Speeding up convolutional neural networks with low rank expansions. *arXiv preprint arXiv:1405.3866*, 2014.
- Mikhail Khodak, Neil Tenenholtz, Lester Mackey, and Nicolò Fusi. Initialization and regularization of factorized neural layers, 2021.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.
- Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding, 2020.
- Brian Lester, Rami Al-Rfou, and Noah Constant. The Power of Scale for Parameter-Efficient Prompt Tuning. *arXiv:2104.08691 [cs]*, April 2021. URL <http://arxiv.org/abs/2104.08691>. arXiv: 2104.08691.
- Chunyuan Li, Heerad Farkhor, Rosanne Liu, and Jason Yosinski. Measuring the Intrinsic Dimension of Objective Landscapes. *arXiv:1804.08838 [cs, stat]*, April 2018a. URL <http://arxiv.org/abs/1804.08838>. arXiv: 1804.08838.
- Xiang Lisa Li and Percy Liang. Prefix-Tuning: Optimizing Continuous Prompts for Generation. *arXiv:2101.00190 [cs]*, January 2021. URL <http://arxiv.org/abs/2101.00190>.
- Yuanzhi Li and Yingyu Liang. Learning overparameterized neural networks via stochastic gradient descent on structured data. In *Advances in Neural Information Processing Systems*, 2018.
- Yuanzhi Li, Yingyu Liang, and Andrej Risteski. Recovery guarantee of weighted low-rank approximation via alternating minimization. In *International Conference on Machine Learning*, pp. 2358–2367. PMLR, 2016.
- Yuanzhi Li, Tengyu Ma, and Hongyang Zhang. Algorithmic regularization in over-parameterized matrix sensing and neural networks with quadratic activations. In *Conference On Learning Theory*, pp. 2–47. PMLR, 2018b.
- Zhaojiang Lin, Andrea Madotto, and Pascale Fung. Exploring versatile generative language model via parameter-efficient transfer learning. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 441–459, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.findings-emnlp.41. URL <https://aclanthology.org/2020.findings-emnlp.41>.

Behrooz Ghorbani, Song Mei, Theodor Misiakiewicz, 和 Andrea Montanari. 神经网络什么时候优于核方法? *arXiv preprint arXiv:2006.13409*, 2020.

Bogdan Gliwa, Iwona Mochol, Maciej Biesek 和 Aleksander Wawer. Samsun 语料库: 用于抽象摘要的人类标注对话数据集. *CoRR*, abs/1911.12237, 2019. 网址 <http://arxiv.org/abs/1911.12237>.

Lars Grasedyck, Daniel Kressner, 和 Christine Tobler. 低秩张量近似技术的文献综述. *GAMM-Mitteilungen*, 36(1):53–78, 2013.

Jihun Ham 和 Daniel D. Lee. 《Grassmann 判别分析: 基于子空间学习的统一视角》。收录于 *ICML*, 第 376–383 页, 2008 年。网址 <https://doi.org/10.1145/1390156.1390204>.

Karen Hambardzumyan, Hrant Khachatrian 和 Jonathan May. WARP: 词级对抗重编程. *arXiv:2101.00121 [cs]*, 2020年12月。网址 <http://arxiv.org/abs/2101.00121>。arXiv: 2101.00121。

何鹏程, 刘晓东, 高建峰, 陈维柱. Deberta: 带有解耦注意力的解码增强BERT, 2021.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan 和 Sylvain Gelly. 用于自然语言处理的参数高效迁移学习. *arXiv:1902.00751 [cs, stat]*, 2019 年 6 月。网址 <http://arxiv.org/abs/1902.00751>.

Max Jaderberg, Andrea Vedaldi 和 Andrew Zisserman. 通过低秩展开加速卷积神经网络. *arXiv preprint arXiv:1405.3866*, 2014.

米哈伊尔·霍达克、尼尔·特能霍尔茨、莱斯特·麦基和尼科洛·富西。分解神经层的初始化与正则化, 2021 年。

Diederik P. Kingma 和 Jimmy Ba. 《Adam: 一种随机优化方法》, 2017 年。

Dmitry Lepikhin, HyukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer 和 Zhifeng Chen. Gshard: 通过条件计算和自动分片扩展巨型模型, 2020年。

Brian Lester, Rami Al-Rfou 和 Noah Constant. 参数高效提示调整的规模威力. *arXiv:2104.08691 [cs]*, 2021年4月。网址 <http://arxiv.org/abs/2104.08691>。arXiv: 2104.08691.

Chunyu Li, Heerad Farkhor, Rosanne Liu 和 Jason Yosinski. 测量目标景观的内在维度. *arXiv:1804.08838 [cs, stat]*, 2018年4月a。网址 <http://arxiv.org/abs/1804.08838>。arXiv: 1804.08838。

Xiang Lisa Li 和 Percy Liang. 前缀调优: 优化连续提示以进行生成. *arXiv:2101.00190 [cs]*, 2021年1月。URL <http://arxiv.org/abs/2101.00190>.

李远志和梁英宇。通过在结构化数据上使用随机梯度下降学习过参数化神经网络。发表于 *Advances in Neural Information Processing Systems*, 2018。

Yuanzhi Li, Yingyu Liang 和 Andrej Risteski. 通过交替最小化的加权低秩近似的恢复保证。刊于 *International Conference on Machine Learning*, 第 2358–2367 页。PMLR, 2016。

李远志, 马腾宇, 张洪阳. 在过参数化矩阵感知和具有二次激活的神经网络中的算法正则化。发表在 *Conference On Learning Theory*, 第 2–47 页。PMLR, 2018b。

林兆江、安德里亚·马多托 和 潘思凯尔·冯。通过参数高效迁移学习探索多功能生成语言模型。发表于 *Findings of the Association for Computational Linguistics: EMNLP 2020*, 第 441–459 页, 在线, 2020 年 11 月。计算语言学协会。doi: 10.18653/v1/2020.findings-emnlp.41。网址 <https://aclanthology.org/2020.findings-emnlp.41>。

-
- Xiao Liu, Yanan Zheng, Zhengxiao Du, Ming Ding, Yujie Qian, Zhilin Yang, and Jie Tang. GPT Understands, Too. *arXiv:2103.10385 [cs]*, March 2021. URL <http://arxiv.org/abs/2103.10385>. arXiv: 2103.10385.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach, 2019.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019.
- Rabeeh Karimi Mahabadi, James Henderson, and Sebastian Ruder. Compacter: Efficient low-rank hypercomplex adapter layers, 2021.
- Linyong Nan, Dragomir Radev, Rui Zhang, Amrit Rau, Abhinand Sivaprasad, Chiachun Hsieh, Xiangru Tang, Aadit Vyas, Neha Verma, Pranav Krishna, et al. Dart: Open-domain structured data record to text generation. *arXiv preprint arXiv:2007.02871*, 2020.
- Jekaterina Novikova, Ondřej Dušek, and Verena Rieser. The e2e dataset: New challenges for end-to-end generation. *arXiv preprint arXiv:1706.09254*, 2017.
- Samet Oymak, Zalan Fabian, Mingchen Li, and Mahdi Soltanolkotabi. Generalization guarantees for neural networks via harnessing the low-rank structure of the jacobian. *arXiv preprint arXiv:1906.05392*, 2019.
- Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. Adapterfusion: Non-destructive task composition for transfer learning, 2021.
- Daniel Povey, Gaofeng Cheng, Yiming Wang, Ke Li, Hainan Xu, Mahsa Yarmohammadi, and Sanjeev Khudanpur. Semi-orthogonal low-rank matrix factorization for deep neural networks. In *Interspeech*, pp. 3743–3747, 2018.
- Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving Language Understanding by Generative Pre-Training. pp. 12, a.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language Models are Unsupervised Multitask Learners. pp. 24, b.
- Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don’t know: Unanswerable questions for squad. *CoRR*, abs/1806.03822, 2018. URL <http://arxiv.org/abs/1806.03822>.
- Sylvestre-Alvise Rebuffi, Hakan Bilen, and Andrea Vedaldi. Learning multiple visual domains with residual adapters. *arXiv:1705.08045 [cs, stat]*, November 2017. URL <http://arxiv.org/abs/1705.08045>. arXiv: 1705.08045.
- Andreas Rücklé, Gregor Geigle, Max Glockner, Tilman Beck, Jonas Pfeiffer, Nils Reimers, and Iryna Gurevych. Adapterdrop: On the efficiency of adapters in transformers, 2020.
- Tara N Sainath, Brian Kingsbury, Vikas Sindhwani, Ebru Arisoy, and Bhuvana Ramabhadran. Low-rank matrix factorization for deep neural network training with high-dimensional output targets. In *2013 IEEE international conference on acoustics, speech and signal processing*, pp. 6655–6659. IEEE, 2013.
- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2020.
- Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pp. 1631–1642, Seattle, Washington, USA, October 2013. Association for Computational Linguistics. URL <https://aclanthology.org/D13-1170>.

小刘、郑亚男、杜政晓、丁明、钱宇杰、杨志林和唐杰。《GPT 也能理解》。
arXiv:2103.10385 [cs], 2021 年 3 月。网址 <http://arxiv.org/abs/2103.10385>。arXiv: 2103.10385。

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer 和 Veselin Stoyanov。Roberta: 一种稳健优化的 BERT 预训练方法, 2019。

Ilya Loshchilov 和 Frank Hutter。解耦权重衰减正则化。*arXiv preprint arXiv:1711.05101*, 2017。

Ilya Loshchilov 和 Frank Hutter。《解耦权重衰减正则化》, 2019 年。

Rabeeh Karimi Mahabadi, James Henderson, 和 Sebastian Ruder。《Compacter: 高效低秩超复数适配器层》, 2021 年。

林永南, Dragomir Radev, 张锐, Amrit Rau, Abhinand Sivaprasad, 谢佳群, 唐祥茹, Aadit Vyas, Neha Verma, Pranav Krishna 等人。Dart: 开放领域结构化数据记录到文本生成。
arXiv preprint arXiv:2007.02871, 2020。

Jekaterina Novikova, Ondřej Dušek 和 Verena Rieser。e2e 数据集: 端到端生成的新挑战。
arXiv preprint arXiv:1706.09254, 2017。Samet Oymak, Zalan Fabian, Mingchen Li 和 Mahdi Soltanolkotabi。通过利用雅可比矩阵的低秩结构为神经网络提供泛化保证。*arXiv preprint arXiv:1906.05392*, 2019。Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho 和 Iryna Gurevych。Adapter-fusion: 非破坏性任务复合的迁移学习方法, 2021。Daniel Povey, Gaofeng Cheng, Yiming Wang, Ke Li, Hainan Xu, Mahsa Yarmohammadi 和 Sanjeev Khudanpur。深度神经网络的半正交低秩矩阵分解。在 *Interspeech*, 第 3743–3747 页, 2018。Alec Radford, Karthik Narasimhan, Tim Salimans 和 Ilya Sutskever。通过生成式预训练提高语言理解能力。第 12 页, a。Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei 和 Ilya Sutskever。语言模型是无监督的多任务学习者。第 24 页, b。Pranav R

Tara N Sainath, Brian Kingsbury, Vikas Sindhwani, Ebru Arisoy 和 Bhuvana Ramabhadran。用于具有高维输出目标的深度神经网络训练的低秩矩阵分解。发表于
2013 IEEE international conference on acoustics, speech and signal processing, 第 6655–6659 页。IEEE, 2013。

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper 和 Bryan Catanzaro。《Megatron-lm: 使用模型并行训练数十亿参数的语言模型》, 2020 年。

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng 和 Christopher Potts。用于情感树库语义组合性的递归深度模型。发表于
Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing, 页 1631–1642, 美国华盛顿州西雅图, 2013 年 10 月。计算语言学协会。网址 <https://aclanthology.org/D13-1170>。

-
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, pp. 6000–6010, 2017.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding, 2019.
- Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. Superglue: A stickier benchmark for general-purpose language understanding systems, 2020.
- Alex Warstadt, Amanpreet Singh, and Samuel R Bowman. Neural network acceptability judgments. *arXiv preprint arXiv:1805.12471*, 2018.
- Adina Williams, Nikita Nangia, and Samuel Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pp. 1112–1122, New Orleans, Louisiana, June 2018. Association for Computational Linguistics. doi: 10.18653/v1/N18-1101. URL <https://www.aclweb.org/anthology/N18-1101>.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, Online, October 2020. Association for Computational Linguistics. URL <https://www.aclweb.org/anthology/2020.emnlp-demos.6>.
- Greg Yang and Edward J. Hu. Feature Learning in Infinite-Width Neural Networks. *arXiv:2011.14522 [cond-mat]*, May 2021. URL <http://arxiv.org/abs/2011.14522>. arXiv: 2011.14522.
- Elad Ben Zaken, Shauli Ravfogel, and Yoav Goldberg. Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models, 2021.
- Yu Zhang, Ekapol Chuangsuwanich, and James Glass. Extracting deep neural network bottleneck features using low-rank matrix factorization. In *2014 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pp. 185–189. IEEE, 2014.
- Yong Zhao, Jinyu Li, and Yifan Gong. Low-rank plus diagonal adaptation for deep neural networks. In *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5005–5009. IEEE, 2016.
- Victor Zhong, Caiming Xiong, and Richard Socher. Seq2sql: Generating structured queries from natural language using reinforcement learning. *CoRR*, abs/1709.00103, 2017. URL <http://arxiv.org/abs/1709.00103>.

A LARGE LANGUAGE MODELS STILL NEED PARAMETER UPDATES

Few-shot learning, or prompt engineering, is very advantageous when we only have a handful of training samples. However, in practice, we can often afford to curate a few thousand or more training examples for performance-sensitive applications. As shown in Table 8, fine-tuning improves the model performance drastically compared to few-shot learning on datasets large and small. We take the GPT-3 few-shot result on RTE from the GPT-3 paper (Brown et al., 2020). For MNLI-matched, we use two demonstrations per class and six in-context examples in total.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, 和 Illia Polosukhin. 《注意力就是你所需要的》。发表于 *Proceedings of the 31st International Conference on Neural Information Processing Systems*, 第 6000–6010 页, 2017 年。

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, 和 Samuel R. Bowman. Glue: 一个用于自然语言理解的多任务基准和分析平台, 2019。

Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, 和 Samuel R. Bowman. 《SuperGLUE: 一种更粘的通用语言理解系统基准》, 2020 年。

Alex Warstadt, Amanpreet Singh 和 Samuel R Bowman. 神经网络可接受性判断。*arXiv preprint arXiv:1805.12471*, 2018。

阿迪娜·威廉姆斯、尼基塔·南吉亚 和 塞缪尔·鲍曼。通过推理进行句子理解的广覆盖挑战语料库。发表于 *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, 第 1112–1122 页, 美国路易斯安那州新奥尔良, 2018 年 6 月。计算语言学协会。doi: 10.18653/v1/N18-1101。网址 <https://www.aclweb.org/anthology/N18-1101>。

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, 和 Alexander M. Rush. 《Transformers: 最先进的自然语言处理》。发表于 *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 第 38–45 页, 在线, 2020 年 10 月。计算语言学协会。网址 <https://www.aclweb.org/anthology/2020.emnlp-demos.6>。

Greg Yang 和 Edward J. Hu. 无限宽度神经网络中的特征学习。*arXiv:2011.14522 [cond-mat]*, 2021 年 5 月。网址 <http://arxiv.org/abs/2011.14522>。arXiv: 2011.14522。

Elad Ben Zaken, Shauli Ravfogel 和 Yoav Goldberg. Bitfit: 基于变压器的掩码语言模型的简单参数高效微调, 2021。

张宇, Ekapol Chuangsuwanich, 和 James Glass. 使用低秩矩阵分解提取深度神经网络瓶颈特征。载于 *2014 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, 第 185–189 页。IEEE, 2014。

赵勇, 李金玉, 龚一凡。深度神经网络的低秩加对角适应。在 *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 第 5005–5009 页。IEEE, 2016 年。

Victor Zhong, Caiming Xiong 和 Richard Socher. 《Seq2SQL: 使用强化学习从自然语言生成结构化查询》。*CoRR*, abs/1709.00103, 2017. 网址 <http://arxiv.org/abs/1709.00103>。

大型语言模型仍然需要参数更新

少样本学习, 或称提示工程, 在我们只有少量训练样本时非常有优势。然而, 在实践中, 对于对性能敏感的应用, 我们通常可以负担得起策划几千个甚至更多的训练示例。如表 8 所示, 与少样本学习相比, 微调在大数据集和小数据集上的模型性能都有显著提升。我们采用了 GPT-3 论文 (Brown 等, 2020) 中 RTE 的 GPT-3 少样本结果。对于 MNLI-matched, 每个类别使用两个示例, 总共使用六个上下文示例。

Method	MNLI-m (Val. Acc./%)	RTE (Val. Acc./%)
GPT-3 Few-Shot	40.6	69.0
GPT-3 Fine-Tuned	89.5	85.4

Table 8: Fine-tuning significantly outperforms few-shot learning on GPT-3 (Brown et al., 2020).

B INFERENCE LATENCY INTRODUCED BY ADAPTER LAYERS

Adapter layers are external modules added to a pre-trained model in a *sequential* manner, whereas our proposal, LoRA, can be seen as external modules added in a parallel manner. Consequently, adapter layers must be computed in addition to the base model, inevitably introducing additional latency. While as pointed out in Rücklé et al. (2020), the latency introduced by adapter layers can be mitigated when the model batch size and/or sequence length is large enough to full utilize the hardware parallelism. We confirm their observation with a similar latency study on GPT-2 medium and point out that there are scenarios, notably online inference where the batch size is small, where the added latency can be significant.

We measure the latency of a single forward pass on an NVIDIA Quadro RTX8000 by averaging over 100 trials. We vary the input batch size, sequence length, and the adapter bottleneck dimension r . We test two adapter designs: the original one by Houlsby et al. (2019), which we call Adapter^H, and a recent, more efficient variant by Lin et al. (2020), which we call Adapter^L. See Section 5.1 for more details on the designs. We plot the slow-down in percentage compared to the no-adapter baseline in Figure 5.

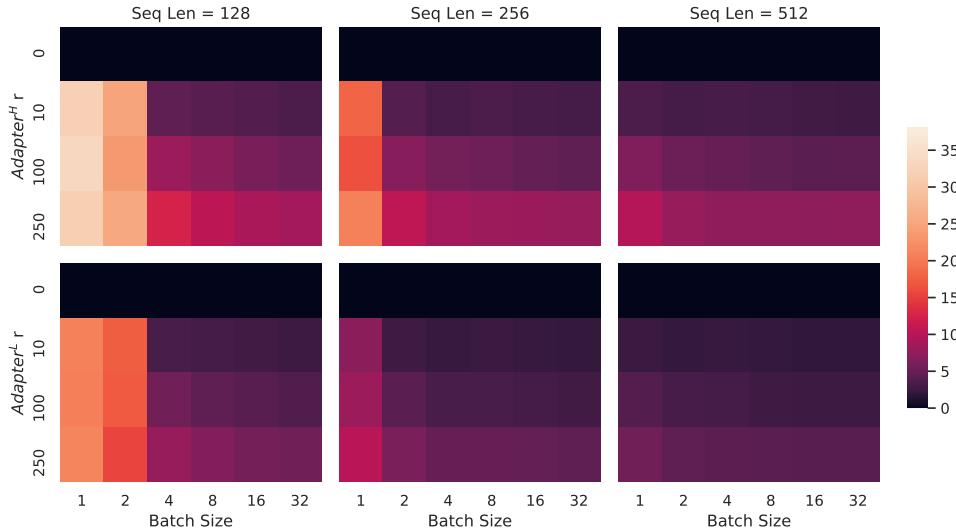


Figure 5: Percentage slow-down of inference latency compared to the no-adapter ($r = 0$) baseline. The top row shows the result for Adapter^H and the bottom row Adapter^L. Larger batch size and sequence length help to mitigate the latency, but the slow-down can be as high as over 30% in an online, short-sequence-length scenario. We tweak the colormap for better visibility.

C DATASET DETAILS

GLUE Benchmark is a wide-ranging collection of natural language understanding tasks. It includes MNLI (inference, Williams et al. (2018)), SST-2 (sentiment analysis, Socher et al. (2013)), MRPC (paraphrase detection, Dolan & Brockett (2005)), CoLA (linguistic acceptability, Warstadt et al. (2018)), QNLI (inference, Rajpurkar et al. (2018)), QQP⁸ (question-answering), RTE (inference),

⁸<https://quoradata.quora.com/First-Quora-Dataset-Release-Question-Pairs>

Method	MNLI-m (Val. Acc./%)	RTE (Val. Acc./%)
GPT-3 Few-Shot	40.6	69.0
GPT-3 Fine-Tuned	89.5	85.4

表8: 在GPT-3 (Brown 等, 2020) 上, 微调的表现显著优于少量样本学习。

B 适配器层引入的推理延迟

适配器层是以 *sequential* 方式添加到预训练模型的外部模块, 而我们提出的 LoRA 可以看作是以并行方式添加的外部模块。因此, 适配器层必须在基础模型之外进行计算, 不可避免地引入额外的延迟。正如 Rücklé 等 (2020) 指出的, 当模型的批量大小和/或序列长度足够大以充分利用硬件并行性时, 适配器层引入的延迟可以得到缓解。我们在 GPT-2 中等模型上通过类似的延迟研究验证了他们的观察, 并指出在某些场景下, 尤其是在批量大小较小的在线推理中, 增加的延迟可能是显著的。

我们通过在 NVIDIA Quadro RTX8000 上平均 100 次试验来测量单次前向传递的延迟。我们改变输入批量大小、序列长度以及适配器瓶颈维度 r 。我们测试了两种适配器设计: Houlsby 等人 (2019) 提出的原始设计, 我们称之为 Adapter^H, 以及 Lin 等人 (2020) 提出的最近更高效的变体, 我们称之为 Adapter^L。关于设计的更多细节, 请参见第 5.1 节。我们在图 5 中绘制了与无适配器基线相比的百分比减慢情况。

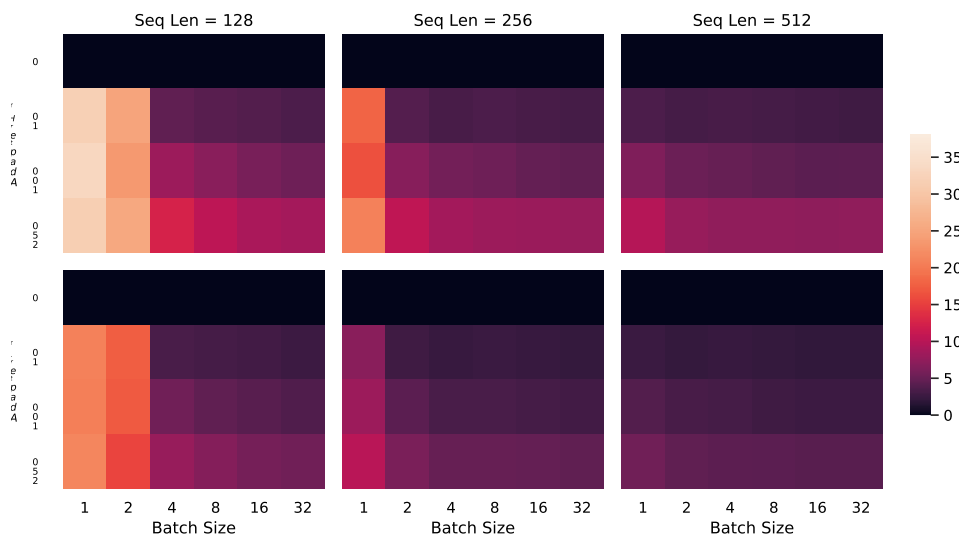


图5: 与无适配器 ($r = 0$) 基线相比, 推理延迟的减速百分比。上行显示 Adapter^H 的结果, 下行显示 Adapter^L 的结果。较大的批量大小和序列长度有助于缓解延迟, 但在在线、短序列长度的场景中, 减速可能高达 30% 以上。我们调整了颜色图以获得更好的可见性。

C 数据集详情

GLUE 基准测试是一个广泛的自然语言理解任务集合。它包括 MNLI (推理, Williams 等, 2018)、SST-2 (情感分析, Socher 等, 2013)、MRPC (同义句检测, Dolan & Brockett, 2005)、CoLA (语言可接受性, Warstadt 等, 2018)、QNLI (推理, Rajpurkar 等, 2018)、QQP⁸ (问答)、RTE (推理)

⁸<https://quoradata.quora.com/First-Quora-Dataset-Release-Question-Pairs>

and STS-B (textual similarity, Cer et al. (2017)). The broad coverage makes GLUE benchmark a standard metric to evaluate NLU models such as RoBERTa and DeBERTa. The individual datasets are released under different permissive licenses.

WikiSQL is introduced in Zhong et al. (2017) and contains 56,355/8,421 training/validation examples. The task is to generate SQL queries from natural language questions and table schemata. We encode context as $x = \{\text{table schema, query}\}$ and target as $y = \{\text{SQL}\}$. The dataset is released under the BSD 3-Clause License.

SAMSum is introduced in Gliwa et al. (2019) and contains 14,732/819 training/test examples. It consists of staged chat conversations between two people and corresponding abstractive summaries written by linguists. We encode context as `"\n"` concatenated utterances followed by a `"\n\n"`, and target as $y = \{\text{summary}\}$. The dataset is released under the non-commercial licence: Creative Commons BY-NC-ND 4.0.

E2E NLG Challenge was first introduced in Novikova et al. (2017) as a dataset for training end-to-end, data-driven natural language generation systems and is commonly used for data-to-text evaluation. The E2E dataset consists of roughly 42,000 training, 4,600 validation, and 4,600 test examples from the restaurant domain. Each source table used as input can have multiple references. Each sample input (x, y) consists of a sequence of slot-value pairs, along with a corresponding natural language reference text. The dataset is released under Creative Commons BY-NC-SA 4.0.

DART is an open-domain data-to-text dataset described in Nan et al. (2020). DART inputs are structured as sequences of ENTITY — RELATION — ENTITY triples. With 82K examples in total, DART is a significantly larger and more complex data-to-text task compared to E2E. The dataset is released under the MIT license.

WebNLG is another commonly used dataset for data-to-text evaluation (Gardent et al., 2017). With 22K examples in total WebNLG comprises 14 distinct categories, nine of which are seen during training. Since five of the 14 total categories are not seen during training, but are represented in the test set, evaluation is typically broken out by “seen” categories (S), “unseen” categories (U) and “all” (A). Each input example is represented by a sequence of SUBJECT — PROPERTY — OBJECT triples. The dataset is released under Creative Commons BY-NC-SA 4.0.

D HYPERPARAMETERS USED IN EXPERIMENTS

D.1 ROBERTA

We train using AdamW with a linear learning rate decay schedule. We sweep learning rate, number of training epochs, and batch size for LoRA. Following Liu et al. (2019), we initialize the LoRA modules to our best MNLI checkpoint when adapting to MRPC, RTE, and STS-B, instead of the usual initialization; the pre-trained model stays frozen for all tasks. We report the median over 5 random seeds; the result for each run is taken from the best epoch. For a fair comparison with the setup in Housley et al. (2019) and Pfeiffer et al. (2021), we restrict the model sequence length to 128 and used a fixed batch size for all tasks. Importantly, we start with the pre-trained RoBERTa large model when adapting to MRPC, RTE, and STS-B, instead of a model already adapted to MNLI. The runs with this restricted setup are marked with †. See the hyperparameters used in our runs in Table 9.

D.2 DEBERTA

We again train using AdamW with a linear learning rate decay schedule. Following He et al. (2021), we tune learning rate, dropout probability, warm-up steps, and batch size. We use the same model sequence length used by (He et al., 2021) to keep our comparison fair. Following He et al. (2021), we initialize the LoRA modules to our best MNLI checkpoint when adapting to MRPC, RTE, and STS-B, instead of the usual initialization; the pre-trained model stays frozen for all tasks. We report the median over 5 random seeds; the result for each run is taken from the best epoch. See the hyperparameters used in our runs in Table 10.

以及 STS-B (文本相似性, Cer 等人, 2017)。广泛的覆盖使 GLUE 基准成为评估 NLU 模型 (如 RoBERTa 和 DeBERTa) 的标准指标。各个数据集以不同的宽松许可发布。

WikiSQL 在 Zhong 等人 (2017) 中被介绍, 其中包含 56,355/8,421 个训练/验证示例。任务是从自然语言问题和表结构生成 SQL 查询。我们将上下文编码为 $x = \{\text{表结构, 查询}\}$, 目标编码为 $y = \{\text{SQL}\}$ 。该数据集以 BSD 3-Clause 许可证发布。

SAMSum 在 Gliwa 等人 (2019) 中提出, 包含 14,732/819 个训练/测试示例。它由两个人之间的分阶段聊天对话以及语言学家撰写的相应摘要组成。我们将上下文编码为 “” \n “” 连接的发言, 后跟 ” \n\n “, 目标编码为 $y = \{\text{summary}\}$ 。该数据集以非商业许可发布: 知识共享署名-非商业性使用-禁止演绎 4.0。

E2E NLG 挑战最初由 Novikova 等人 (2017) 提出, 作为一个用于训练端到端、数据驱动的自然语言生成系统的数据集, 并且常用于数据到文本的评估。E2E 数据集大约包含 42,000 个训练样本、4,600 个验证样本和 4,600 个测试样本, 均来自餐厅领域。每个用作输入的源表可以有多个参考文本。每个样本输入由一系列槽值对组成, 并附有相应的自然语言参考文本。该数据集在知识共享署名-非商业性使用-相同方式共享 4.0 国际许可协议下发布。

DART 是一个开放领域的数据到文本的数据集, 由 Nan 等人 (2020 年) 描述。DART 的输入结构为实体 — 关系 — 实体三元组序列。DART 总共有 82K 个示例, 与 E2E 相比, 它是一个更大且更复杂的数据到文本任务。该数据集在 MIT 许可证下发布。

WebNLG 是另一个常用的数据到文本评估数据集 (Gardent 等, 2017)。WebNLG 总共包含 22K 个示例, 涵盖 14 个不同的类别, 其中九个在训练过程中出现过。由于总共 14 个类别中有五个在训练期间未出现, 但在测试集中存在, 因此评估通常按 “已见” 类别 (S)、“未见” 类别 (U) 和 “全部” (A) 进行拆分。每个输入示例由一系列主体 — 属性 — 客体三元组表示。该数据集以知识共享署名-非商业性使用-相同方式共享 4.0 (Creative Commons BY-NC-SA 4.0) 许可发布。

D 实验中使用的超参数

D.1 罗伯塔

我们使用 AdamW 并配合线性学习率衰减计划进行训练。我们对 LoRA 的学习率、训练轮数和批量大小进行搜索。遵循 Liu 等人 (2019) 的方法, 在适应 MRPC、RTE 和 STS-B 时, 我们将 LoRA 模块初始化为我们最佳的 MNLI 检查点, 而不是通常的初始化方式; 预训练模型在所有任务中保持冻结。我们报告 5 个随机种子的中位数; 每次运行的结果取自最佳轮次。为了与 Houlsby 等人 (2019) 和 Pfeiffer 等人 (2021) 的设置进行公平比较, 我们将模型序列长度限制为 128, 并且为所有任务使用固定批量大小。重要的是, 在适应 MRPC、RTE 和 STS-B 时, 我们从预训练的 RoBERTa large 模型开始, 而不是已经适应 MNLI 的模型。使用此限制设置的运行标记为 †。我们运行中使用的超参数见表 9。

D.2 DEBERTA

我们再次使用 AdamW 进行训练, 并采用线性学习率衰减策略。按照 He 等人 (2021) 的做法, 我们调整学习率、dropout 概率、预热步骤数和批量大小。为了保持比较的公平性, 我们使用与 (He 等人, 2021) 相同的模型序列长度。按照 He 等人 (2021) 的做法, 在将 LoRA 模块适配到 MRPC、RTE 和 STS-B 时, 我们将其初始化为我们在 MNLI 上的最佳检查点, 而不是通常的初始化; 预训练模型在所有任务中保持冻结状态。我们报告 5 个随机种子的中位数; 每次运行的结果取自最佳的训练轮次。有关我们实验中使用的超参数, 请参见表 10。

Method	Dataset	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B
	Optimizer	AdamW							
	Warmup Ratio	0.06							
	LR Schedule	Linear							
RoBERTa base LoRA	Batch Size	16	16	16	32	32	16	32	16
	# Epochs	30	60	30	80	25	25	80	40
	Learning Rate	5E-04	5E-04	4E-04	4E-04	4E-04	5E-04	5E-04	4E-04
	LoRA Config.	$r_q = r_v = 8$							
	LoRA α	8							
	Max Seq. Len.	512							
RoBERTa large LoRA	Batch Size	4	4	4	4	4	4	8	8
	# Epochs	10	10	20	20	10	20	20	30
	Learning Rate	3E-04	4E-04	3E-04	2E-04	2E-04	3E-04	4E-04	2E-04
	LoRA Config.	$r_q = r_v = 8$							
	LoRA α	16							
	Max Seq. Len.	128	128	512	128	512	512	512	512
RoBERTa large LoRA [†]	Batch Size	4							
	# Epochs	10	10	20	20	10	20	20	10
	Learning Rate	3E-04	4E-04	3E-04	2E-04	2E-04	3E-04	4E-04	2E-04
	LoRA Config.	$r_q = r_v = 8$							
	LoRA α	16							
	Max Seq. Len.	128							
RoBERTa large Adpt ^P (3M) [†]	Batch Size	32							
	# Epochs	10	20	20	20	10	20	20	20
	Learning Rate	3E-05	3E-05	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04
	Bottleneck r	64							
	Max Seq. Len.	128							
RoBERTa large Adpt ^P (0.8M) [†]	Batch Size	32							
	# Epochs	5	20	20	20	10	20	20	20
	Learning Rate	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04
	Bottleneck r	16							
	Max Seq. Len.	128							
RoBERTa large Adpt ^H (6M) [†]	Batch Size	32							
	# Epochs	10	5	10	10	5	20	20	10
	Learning Rate	3E-05	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04
	Bottleneck r	64							
	Max Seq. Len.	128							
RoBERTa large Adpt ^H (0.8M) [†]	Batch Size	32							
	# Epochs	10	5	10	10	5	20	20	10
	Learning Rate	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04
	Bottleneck r	8							
	Max Seq. Len.	128							

Table 9: The hyperparameters we used for RoBERTa on the GLUE benchmark.

D.3 GPT-2

We train all of our GPT-2 models using AdamW (Loshchilov & Hutter, 2017) with a linear learning rate schedule for 5 epochs. We use the batch size, learning rate, and beam search beam size described in Li & Liang (2021). Accordingly, we also tune the above hyperparameters for LoRA. We report the mean over 3 random seeds; the result for each run is taken from the best epoch. The hyperparameters used for LoRA in GPT-2 are listed in Table 11. For those used for other baselines, see Li & Liang (2021).

D.4 GPT-3

For all GPT-3 experiments, we train using AdamW (Loshchilov & Hutter, 2017) for 2 epochs with a batch size of 128 samples and a weight decay factor of 0.1. We use a sequence length of 384 for

Method	Dataset	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B
	Optimizer	AdamW							
	Warmup Ratio	0.06							
	LR Schedule	Linear							
RoBERTa base LoRA	Batch Size	16	16	16	32	32	16	32	16
	# Epochs	30	60	30	80	25	25	80	40
	Learning Rate	5E-04	5E-04	4E-04	4E-04	4E-04	5E-04	5E-04	4E-04
	LoRA Config.	$r_q = r_v = 8$							
	LoRA α	8							
	Max Seq. Len.	512							
RoBERTa large LoRA	Batch Size	4	4	4	4	4	4	8	8
	# Epochs	10	10	20	20	10	20	20	30
	Learning Rate	3E-04	4E-04	3E-04	2E-04	2E-04	3E-04	4E-04	2E-04
	LoRA Config.	$r_q = r_v = 8$							
	LoRA α	16							
	Max Seq. Len.	128	128	512	128	512	512	512	512
RoBERTa large LoRA [†]	Batch Size	4							
	# Epochs	10	10	20	20	10	20	20	10
	Learning Rate	3E-04	4E-04	3E-04	2E-04	2E-04	3E-04	4E-04	2E-04
	LoRA Config.	$r_q = r_v = 8$							
	LoRA α	16							
	Max Seq. Len.	128							
RoBERTa large Adpt ^P (3M) [†]	Batch Size	32							
	# Epochs	10	20	20	20	10	20	20	20
	Learning Rate	3E-05	3E-05	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04
	Bottleneck r	64							
	Max Seq. Len.	128							
RoBERTa large Adpt ^P (0.8M) [†]	Batch Size	32							
	# Epochs	5	20	20	20	10	20	20	20
	Learning Rate	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04
	Bottleneck r	16							
	Max Seq. Len.	128							
RoBERTa large Adpt ^H (6M) [†]	Batch Size	32							
	# Epochs	10	5	10	10	5	20	20	10
	Learning Rate	3E-05	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04
	Bottleneck r	64							
	Max Seq. Len.	128							
RoBERTa large Adpt ^H (0.8M) [†]	Batch Size	32							
	# Epochs	10	5	10	10	5	20	20	10
	Learning Rate	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04
	Bottleneck r	8							
	Max Seq. Len.	128							

表 9: T 我们在 GLUE 基准上用于 RoBERTa 的超参数

hmark。

D.3 GPT-2

我们使用 AdamW (Loshchilov & Hutter, 2017) 训练所有的 GPT-2 模型，并采用线性学习率调度进行 5 个周期的训练。我们使用 Li & Liang (2021) 中描述的批量大小、学习率和束搜索束大小。因此，我们也对 LoRA 调整上述超参数。我们报告 3 个随机种子的平均值；每次运行的结果取自最佳周期。GPT-2 中 LoRA 使用的超参数列在表 11 中。其他基线使用的超参数请参见 Li & Liang (2021)。

D.4 GPT-3

对于所有 GPT-3 实验，我们使用 AdamW (Loshchilov & Hutter, 2017) 训练 2 个周期，批量大小为 128 个样本，权重衰减系数为 0.1。我们使用的序列长度为 384。

Method	Dataset	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B
DeBERTa XXL LoRA	Optimizer	AdamW							
	Warmup Ratio	0.1							
	LR Schedule	Linear							
	Batch Size	8	8	32	4	6	8	4	4
	# Epochs	5	16	30	10	8	11	11	10
	Learning Rate	1E-04	6E-05	2E-04	1E-04	1E-04	1E-04	2E-04	2E-04
	Weight Decay	0	0.01	0.01	0	0.01	0.01	0.01	0.1
	CLS Dropout	0.15	0	0	0.1	0.1	0.2	0.2	0.2
LoRA Config.		$r_q = r_v = 8$							
LoRA α		8							
Max Seq. Len.		256	128	128	64	512	320	320	128

Table 10: The hyperparameters for DeBERTa XXL on tasks included in the GLUE benchmark.

Dataset	E2E	WebNLG	DART
	Training		
Optimizer		AdamW	
Weight Decay	0.01	0.01	0.0
Dropout Prob	0.1	0.1	0.0
Batch Size		8	
# Epoch		5	
Warmup Steps		500	
Learning Rate Schedule		Linear	
Label Smooth	0.1	0.1	0.0
Learning Rate		0.0002	
Adaptation		$r_q = r_v = 4$	
LoRA α		32	
	Inference		
Beam Size		10	
Length Penalty	0.9	0.8	0.8
no repeat ngram size		4	

Table 11: The hyperparameters for GPT-2 LoRA on E2E, WebNLG and DART.

WikiSQL (Zhong et al., 2017), 768 for MNLI (Williams et al., 2018), and 2048 for SAMSum (Gliwa et al., 2019). We tune learning rate for all method-dataset combinations. See Section D.4 for more details on the hyperparameters used. For prefix-embedding tuning, we find the optimal l_p and l_i to be 256 and 8, respectively, totalling 3.2M trainable parameters. We use $l_p = 8$ and $l_i = 8$ for prefix-layer tuning with 20.2M trainable parameters to obtain the overall best performance. We present two parameter budgets for LoRA: 4.7M ($r_q = r_v = 1$ or $r_v = 2$) and 37.7M ($r_q = r_v = 8$ or $r_q = r_k = r_v = r_o = 2$). We report the best validation performance from each run. The training hyperparameters used in our GPT-3 experiments are listed in Table 12.

E COMBINING LORA WITH PREFIX TUNING

LoRA can be naturally combined with existing prefix-based approaches. In this section, we evaluate two combinations of LoRA and variants of prefix-tuning on WikiSQL and MNLI.

LoRA+PrefixEmbed (LoRA+PE) combines LoRA with prefix-embedding tuning, where we insert $l_p + l_i$ special tokens whose embeddings are treated as trainable parameters. For more on prefix-embedding tuning, see Section 5.1.

LoRA+PrefixLayer (LoRA+PL) combines LoRA with prefix-layer tuning. We also insert $l_p + l_i$ special tokens; however, instead of letting the hidden representations of these tokens evolve natu-

Method	Dataset	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B
DeBERTa XXL LoRA	Optimizer	AdamW							
	Warmup Ratio	0.1							
	LR Schedule	Linear							
	Batch Size	8	8	32	4	6	8	4	4
	# Epochs	5	16	30	10	8	11	11	10
	Learning Rate	1E-04	6E-05	2E-04	1E-04	1E-04	1E-04	2E-04	2E-04
	Weight Decay	0	0.01	0.01	0	0.01	0.01	0.01	0.1
	CLS Dropout	0.15	0	0	0.1	0.1	0.2	0.2	0.2
LoRA Config.		$r_q = r_v = 8$							
LoRA α		8							
Max Seq. Len.		256	128	128	64	512	320	320	128

表 10: 用于 GLUE 基准任务中 DeBERTa XXL 的超参数。

Dataset	E2E	WebNLG	DART
	Training		
Optimizer		AdamW	
Weight Decay	0.01	0.01	0.0
Dropout Prob	0.1	0.1	0.0
Batch Size		8	
# Epoch		5	
Warmup Steps		500	
Learning Rate Schedule		Linear	
Label Smooth	0.1	0.1	0.0
Learning Rate		0.0002	
Adaptation		$r_q = r_v = 4$	
LoRA α		32	
	Inference		
Beam Size		10	
Length Penalty	0.9	0.8	0.8
no repeat ngram size		4	

表11: E2E、WebNLG和DART上GPT-2 LoRA的超参数。

WikiSQL (Zhong 等, 2017), MNLI 为 768 (Williams 等, 2018), SAMSum 为 2048 (Gliwa 等, 2019)。我们为所有方法-数据集组合调整学习率。更多使用的超参数详情请参见第 D.4 节。在前缀嵌入调优中, 我们发现最优的 l_p 和 l_i 分别为 256 和 8, 总计 3.2M 个可训练参数。我们在前缀层调优中使用 $l_p = 8$ 和 $l_i = 8$, 具有 20.2M 个可训练参数, 以获得整体最佳性能。我们为 LoRA 提供两个参数预算: 4.7M ($r_q = r_v = 1$ 或 $r_v = 2$) 和 37.7M ($r_q = r_v = 8$ 或 $r_q = r_k = r_v = r_o = 2$)。我们报告每次运行的最佳验证性能。我们在 GPT-3 实验中使用的训练超参数列于表 12 中。

将 LoRA 与前缀微调结合

LoRA 可以自然地与现有的基于前缀的方法结合。在本节中, 我们评估了 LoRA 与前缀调优变体在 WikiSQL 和 MNLI 上的两种组合。

LoRA+PrefixEmbed (LoRA+PE) 将 LoRA 与前缀嵌入调优结合起来, 其中我们插入 $l_p + l_i$ 个特殊标记, 其嵌入被视为可训练参数。有关前缀嵌入调优的更多内容, 请参见第 5.1 节。

LoRA+预置层 (LoRA+PL) 将 LoRA 与预置层微调结合起来。我们还插入了 $l_p + l_i$ 个特殊标记; 然而, 并不是让这些标记的隐藏表示自然演变

Hyperparameters	Fine-Tune	PreEmbed	PreLayer	BitFit	Adapter ^H	LoRA
Optimizer			AdamW			
Batch Size			128			
# Epoch			2			
Warmup Tokens			250,000			
LR Schedule			Linear			
Learning Rate	5.00E-06	5.00E-04	1.00E-04	1.6E-03	1.00E-04	2.00E-04

Table 12: The training hyperparameters used for different GPT-3 adaption methods. We use the same hyperparameters for all datasets after tuning learning rate.

rally, we replace them after every Transformer block with an input agnostic vector. Thus, both the embeddings and subsequent Transformer block activations are treated as trainable parameters. For more on prefix-layer tuning, see Section 5.1.

In Table 15, we show the evaluation results of LoRA+PE and LoRA+PL on WikiSQL and MultiNLI. First of all, LoRA+PE significantly outperforms both LoRA and prefix-embedding tuning on WikiSQL, which indicates that LoRA is somewhat orthogonal to prefix-embedding tuning. On MultiNLI, the combination of LoRA+PE doesn’t perform better than LoRA, possibly because LoRA on its own already achieves performance comparable to the human baseline. Secondly, we notice that LoRA+PL performs slightly worse than LoRA even with more trainable parameters. We attribute this to the fact that prefix-layer tuning is very sensitive to the choice of learning rate and thus makes the optimization of LoRA weights more difficult in LoRA+PL.

F ADDITIONAL EMPIRICAL EXPERIMENTS

F.1 ADDITIONAL EXPERIMENTS ON GPT-2

We also repeat our experiment on DART (Nan et al., 2020) and WebNLG (Gardent et al., 2017) following the setup of Li & Liang (2021). The result is shown in Table 13. Similar to our result on E2E NLG Challenge, reported in Section 5, LoRA performs better than or at least on-par with prefix-based approaches given the same number of trainable parameters.

Method	# Trainable Parameters	BLEU↑	DART MET↑	TER↓
GPT-2 Medium				
Fine-Tune	354M	46.2	0.39	0.46
Adapter ^L	0.37M	42.4	0.36	0.48
Adapter ^L	11M	45.2	0.38	0.46
FT ^{Top2}	24M	41.0	0.34	0.56
PrefLayer	0.35M	46.4	0.38	0.46
LoRA	0.35M	47.1_{±.2}	0.39	0.46
GPT-2 Large				
Fine-Tune	774M	47.0	0.39	0.46
Adapter ^L	0.88M	45.7 _{±.1}	0.38	0.46
Adapter ^L	23M	47.1 _{±.1}	0.39	0.45
PrefLayer	0.77M	46.7	0.38	0.45
LoRA	0.77M	47.5_{±.1}	0.39	0.45

Table 13: GPT-2 with different adaptation methods on DART. The variances of MET and TER are less than 0.01 for all adaption approaches.

Hyperparameters	Fine-Tune	PreEmbed	PreLayer	BitFit	Adapter ^H	LoRA
Optimizer	AdamW					
Batch Size	128					
# Epoch	2					
Warmup Tokens	250,000					
LR Schedule	Linear					
Learning Rate	5.00E-06	5.00E-04	1.00E-04	1.6E-03	1.00E-04	2.00E-04

表12: 不同GPT-3适应方法使用的训练超参数。在调整学习率后, 我们对所有数据集使用相同的超参数。

总体上, 我们在每个 Transformer 块之后用一个与输入无关的向量替换它们。因此, 嵌入和随后的 Transformer 块激活都被视为可训练的参数。有关前缀层调优的更多信息, 请参见第 5.1 节。

在表15中, 我们展示了LoRA+PE和LoRA+PL在WikiSQL和MultiNLI上的评估结果。首先, LoRA+PE在WikiSQL上明显优于LoRA和前缀嵌入调优, 这表明LoRA在某种程度上与前缀嵌入调优是正交的。在MultiNLI上, LoRA+PE的组合并没有比LoRA表现更好, 这可能是由于LoRA本身已经达到了与人类基线相当的性能。其次, 我们注意到即使有更多可训练参数, LoRA+PL的表现仍略逊于LoRA。我们认为这是因为前缀层调优对学习率的选择非常敏感, 因此使得LoRA权重在LoRA+PL中的优化更加困难。

附加实证实验

F.1 关于 GPT-2 的附加实验

我们还按照 Li & Liang (2021) 的设置, 在 DART (Nan et al., 2020) 和 WebNLG (Gardent et al., 2017) 上重复了我们的实验。结果如表 13 所示。与我们在 E2E NLG Challenge 上的结果类似 (见第 5 节), 在相同数量的可训练参数下, LoRA 的表现优于或至少与基于前缀的方法持平。

Method	# Trainable Parameters	BLEU↑	DART MET↑	TER↓
GPT-2 Medium				
Fine-Tune	354M	46.2	0.39	0.46
Adapter ^L	0.37M	42.4	0.36	0.48
Adapter ^L	11M	45.2	0.38	0.46
FT ^{Top2}	24M	41.0	0.34	0.56
PrefLayer	0.35M	46.4	0.38	0.46
LoRA	0.35M	47.1_{±.2}	0.39	0.46
GPT-2 Large				
Fine-Tune	774M	47.0	0.39	0.46
Adapter ^L	0.88M	45.7 _{±.1}	0.38	0.46
Adapter ^L	23M	47.1 _{±.1}	0.39	0.45
PrefLayer	0.77M	46.7	0.38	0.45
LoRA	0.77M	47.5_{±.1}	0.39	0.45

表13: 在DART上使用不同适应方法的GPT-2。所有适应方法的MET和TER方差均小于0.01。

Method	WebNLG								
	U	BLEU↑ S	A	U	MET↑ S	A	U	TER↓ S	A
GPT-2 Medium									
Fine-Tune (354M)	27.7	64.2	46.5	.30	.45	.38	.76	.33	.53
Adapter ^L (0.37M)	45.1	54.5	50.2	.36	.39	.38	.46	.40	.43
Adapter ^L (11M)	48.3	60.4	54.9	.38	.43	.41	.45	.35	.39
FT ^{Top2} (24M)	18.9	53.6	36.0	.23	.38	.31	.99	.49	.72
Prefix (0.35M)	45.6	62.9	55.1	.38	.44	.41	.49	.35	.40
LoRA (0.35M)	46.7 _{±.4}	62.1 _{±.2}	55.3_{±.2}	.38	.44	.41	.46	.33	.39
GPT-2 Large									
Fine-Tune (774M)	43.1	65.3	55.5	.38	.46	.42	.53	.33	.42
Adapter ^L (0.88M)	49.8_{±.0}	61.1 _{±.0}	56.0 _{±.0}	.38	.43	.41	.44	.35	.39
Adapter ^L (23M)	49.2 _{±.1}	64.7 _{±.2}	57.7_{±.1}	.39	.46	.43	.46	.33	.39
Prefix (0.77M)	47.7	63.4	56.3	.39	.45	.42	.48	.34	.40
LoRA (0.77M)	48.4 _{±.3}	64.0 _{±.3}	57.0 _{±.1}	.39	.45	.42	.45	.32	.38

Table 14: GPT-2 with different adaptation methods on WebNLG. The variances of MET and TER are less than 0.01 for all the experiments we ran. “U” indicates unseen categories, “S” indicates seen categories, and “A” indicates all categories in the test set of WebNLG.

F.2 ADDITIONAL EXPERIMENTS ON GPT-3

We present additional runs on GPT-3 with different adaptation methods in Table 15. The focus is on identifying the trade-off between performance and the number of trainable parameters.

F.3 LOW-DATA REGIME

To evaluate the performance of different adaptation approaches in the low-data regime, we randomly sample 100, 1k and 10k training examples from the full training set of MNLI to form the low-data MNLI- n tasks. In Table 16, we show the performance of different adaptation approaches on MNLI- n . To our surprise, PrefixEmbed and PrefixLayer performs very poorly on MNLI-100 dataset, with PrefixEmbed performing only slightly better than random chance (37.6% vs. 33.3%). PrefixLayer performs better than PrefixEmbed but is still significantly worse than Fine-Tune or LoRA on MNLI-100. The gap between prefix-based approaches and LoRA/Fine-tuning becomes smaller as we increase the number of training examples, which might suggest that prefix-based approaches are not suitable for low-data tasks in GPT-3. LoRA achieves better performance than fine-tuning on both MNLI-100 and MNLI-Full, and comparable results on MNLI-1k and MNLI-10K considering the (± 0.3) variance due to random seeds.

The training hyperparameters of different adaptation approaches on MNLI- n are reported in Table 17. We use a smaller learning rate for PrefixLayer on the MNLI-100 set, as the training loss does not decrease with a larger learning rate.

G MEASURING SIMILARITY BETWEEN SUBSPACES

In this paper we use the measure $\phi(A, B, i, j) = \psi(U_A^i, U_B^j) = \frac{\|U_A^{i\top} U_B^j\|_F^2}{\min_{\{i,j\}}}$ to measure the subspace similarity between two column orthonormal matrices $U_A^i \in \mathbb{R}^{d \times i}$ and $U_B^j \in \mathbb{R}^{d \times j}$, obtained by taking columns of the left singular matrices of A and B . We point out that this similarity is simply a reverse of the standard Projection Metric that measures distance between subspaces Ham & Lee (2008).

Method	WebNLG								
	U	BLEU $\uparrow$ S	A	U	MET $\uparrow$ S	A	U	TER $\downarrow$ S	A
GPT-2 Medium									
Fine-Tune (354M)	27.7	64.2	46.5	.30	.45	.38	.76	.33	.53
Adapter ^L (0.37M)	45.1	54.5	50.2	.36	.39	.38	.46	.40	.43
Adapter ^L (11M)	48.3	60.4	54.9	.38	.43	.41	.45	.35	.39
FT ^{Top2} (24M)	18.9	53.6	36.0	.23	.38	.31	.99	.49	.72
Prefix (0.35M)	45.6	62.9	55.1	.38	.44	.41	.49	.35	.40
LoRA (0.35M)	46.7 $\pm$.4	62.1 $\pm$.2	55.3$\pm$.2	.38	.44	.41	.46	.33	.39
GPT-2 Large									
Fine-Tune (774M)	43.1	65.3	55.5	.38	.46	.42	.53	.33	.42
Adapter ^L (0.88M)	49.8$\pm$.0	61.1 $\pm$.0	56.0 $\pm$.0	.38	.43	.41	.44	.35	.39
Adapter ^L (23M)	49.2 $\pm$.1	64.7 $\pm$.2	57.7$\pm$.1	.39	.46	.43	.46	.33	.39
Prefix (0.77M)	47.7	63.4	56.3	.39	.45	.42	.48	.34	.40
LoRA (0.77M)	48.4 $\pm$.3	64.0 $\pm$.3	57.0 $\pm$.1	.39	.45	.42	.45	.32	.38

表 14: 使用不同适应方法的 GPT-2 在 WebNLG 上的表现。我们进行的所有实验中, MET 和 TER 的方差均小于 0.01。“U”表示未见过的类别,“S”表示已见过的类别,“A”表示 WebNLG 测试集中的所有类别。

F.2 关于 GPT-3 的附加实验

我们在表15中展示了使用不同适应方法的GPT-3的额外运行结果。重点是识别性能与可训练参数数量之间的权衡。

F.3 低数据模式

为了评估不同适应方法在低数据量环境下的性能,我们从完整的 MNLI 训练集中随机抽取 100、1k 和 10k 个训练样本,形成低数据 MNLI- n 任务。在表 16 中,我们展示了不同适应方法在 MNLI- n 上的性能。令我们惊讶的是,PrefixEmbed 和 PrefixLayer 在 MNLI-100 数据集上的表现非常差,其中 PrefixEmbed 的表现仅略好于随机猜测 (37.6% 对比 33.3%)。PrefixLayer 的表现优于 PrefixEmbed,但仍明显逊色于 MNLI-100 上的微调 (Fine-Tune) 或 LoRA。随着训练样本数量的增加,基于前缀的方法与 LoRA/微调之间的差距变小,这可能表明基于前缀的方法不适合 GPT-3 的低数据任务。LoRA 在 MNLI-100 和 MNLI-Full 上的表现优于微调,并且在 MNLI-1k 和 MNLI-10k 上的结果可与微调相媲美,考虑到由于随机种子带来的 (± 0.3) 方差。

不同适应方法在 MNLI- n 上的训练超参数在表 17 中报告。我们在 MNLI-100 集上对 PrefixLayer 使用较小的学习率,因为使用较大学习率时训练损失不会下降。

G 测量子空间之间的相似性

在本文中,我们使用度量 $\phi(A, B, i, j) = \psi(U_A^i, U_B^j) = \frac{\|U_A^i - U_B^j\|_F^2}{\min\{i, j\}}$ 来衡量两个列正交矩阵 $U_A^i \in \mathbb{R}^{d \times i}$ 和 $U_B^j \in \mathbb{R}^{d \times j}$ 之间的子空间相似性,这些矩阵是通过取 A 和 B 的左奇异矩阵的列获得的。我们指出,这种相似性只是标准投影度量的反向,该度量用于衡量子空间之间的距离 (Ham & Lee, 2008)。

Method	Hyperparameters	# Trainable Parameters	WikiSQL	MNLI-m
Fine-Tune	-	175B	73.8	89.5
PrefixEmbed	$l_p = 32, l_i = 8$	0.4 M	55.9	84.9
	$l_p = 64, l_i = 8$	0.9 M	58.7	88.1
	$l_p = 128, l_i = 8$	1.7 M	60.6	88.0
	$l_p = 256, l_i = 8$	3.2 M	63.1	88.6
	$l_p = 512, l_i = 8$	6.4 M	55.9	85.8
PrefixLayer	$l_p = 2, l_i = 2$	5.1 M	68.5	89.2
	$l_p = 8, l_i = 0$	10.1 M	69.8	88.2
	$l_p = 8, l_i = 8$	20.2 M	70.1	89.5
	$l_p = 32, l_i = 4$	44.1 M	66.4	89.6
	$l_p = 64, l_i = 0$	76.1 M	64.9	87.9
Adapter ^H	$r = 1$	7.1 M	71.9	89.8
	$r = 4$	21.2 M	73.2	91.0
	$r = 8$	40.1 M	73.2	91.5
	$r = 16$	77.9 M	73.2	91.5
	$r = 64$	304.4 M	72.6	91.5
LoRA	$r_v = 2$	4.7 M	73.4	91.7
	$r_q = r_v = 1$	4.7 M	73.4	91.3
	$r_q = r_v = 2$	9.4 M	73.3	91.4
	$r_q = r_k = r_v = r_o = 1$	9.4 M	74.1	91.2
	$r_q = r_v = 4$	18.8 M	73.7	91.3
	$r_q = r_k = r_v = r_o = 2$	18.8 M	73.7	91.7
	$r_q = r_v = 8$	37.7 M	73.8	91.6
	$r_q = r_k = r_v = r_o = 4$	37.7 M	74.0	91.7
	$r_q = r_v = 64$	301.9 M	73.6	91.4
	$r_q = r_k = r_v = r_o = 64$	603.8 M	73.9	91.4
LoRA+PE	$r_q = r_v = 8, l_p = 8, l_i = 4$	37.8 M	75.0	91.4
	$r_q = r_v = 32, l_p = 8, l_i = 4$	151.1 M	75.9	91.1
	$r_q = r_v = 64, l_p = 8, l_i = 4$	302.1 M	76.2	91.3
LoRA+PL	$r_q = r_v = 8, l_p = 8, l_i = 4$	52.8 M	72.9	90.2

Table 15: Hyperparameter analysis of different adaptation approaches on WikiSQL and MNLI. Both prefix-embedding tuning (PrefixEmbed) and prefix-layer tuning (PrefixLayer) perform worse as we increase the number of trainable parameters, while LoRA’s performance stabilizes. Performance is measured in validation accuracy.

Method	MNLI(m)-100	MNLI(m)-1k	MNLI(m)-10k	MNLI(m)-392K
GPT-3 (Fine-Tune)	60.2	85.8	88.9	89.5
GPT-3 (PrefixEmbed)	37.6	75.2	79.5	88.6
GPT-3 (PrefixLayer)	48.3	82.5	85.9	89.6
GPT-3 (LoRA)	63.8	85.6	89.2	91.7

Table 16: Validation accuracy of different methods on subsets of MNLI using GPT-3 175B. MNLI- n describes a subset with n training examples. We evaluate with the full validation set. LoRA performs exhibits favorable sample-efficiency compared to other methods, including fine-tuning.

To be concrete, let the singular values of $U_A^{i\top} U_B^j$ to be $\sigma_1, \sigma_2, \dots, \sigma_p$ where $p = \min\{i, j\}$. We know that the Projection Metric Ham & Lee (2008) is defined as:

$$d(U_A^i, U_B^j) = \sqrt{p - \sum_{i=1}^p \sigma_i^2} \in [0, \sqrt{p}]$$

Method	Hyperparameters	# Trainable Parameters	WikiSQL	MNLI-m
Fine-Tune	-	175B	73.8	89.5
PrefixEmbed	$l_p = 32, l_i = 8$	0.4 M	55.9	84.9
	$l_p = 64, l_i = 8$	0.9 M	58.7	88.1
	$l_p = 128, l_i = 8$	1.7 M	60.6	88.0
	$l_p = 256, l_i = 8$	3.2 M	63.1	88.6
	$l_p = 512, l_i = 8$	6.4 M	55.9	85.8
PrefixLayer	$l_p = 2, l_i = 2$	5.1 M	68.5	89.2
	$l_p = 8, l_i = 0$	10.1 M	69.8	88.2
	$l_p = 8, l_i = 8$	20.2 M	70.1	89.5
	$l_p = 32, l_i = 4$	44.1 M	66.4	89.6
	$l_p = 64, l_i = 0$	76.1 M	64.9	87.9
Adapter ^H	$r = 1$	7.1 M	71.9	89.8
	$r = 4$	21.2 M	73.2	91.0
	$r = 8$	40.1 M	73.2	91.5
	$r = 16$	77.9 M	73.2	91.5
	$r = 64$	304.4 M	72.6	91.5
LoRA	$r_v = 2$	4.7 M	73.4	91.7
	$r_q = r_v = 1$	4.7 M	73.4	91.3
	$r_q = r_v = 2$	9.4 M	73.3	91.4
	$r_q = r_k = r_v = r_o = 1$	9.4 M	74.1	91.2
	$r_q = r_v = 4$	18.8 M	73.7	91.3
	$r_q = r_k = r_v = r_o = 2$	18.8 M	73.7	91.7
	$r_q = r_v = 8$	37.7 M	73.8	91.6
	$r_q = r_k = r_v = r_o = 4$	37.7 M	74.0	91.7
	$r_q = r_v = 64$	301.9 M	73.6	91.4
	$r_q = r_k = r_v = r_o = 64$	603.8 M	73.9	91.4
LoRA+PE	$r_q = r_v = 8, l_p = 8, l_i = 4$	37.8 M	75.0	91.4
	$r_q = r_v = 32, l_p = 8, l_i = 4$	151.1 M	75.9	91.1
	$r_q = r_v = 64, l_p = 8, l_i = 4$	302.1 M	76.2	91.3
LoRA+PL	$r_q = r_v = 8, l_p = 8, l_i = 4$	52.8 M	72.9	90.2

表15: 不同适应方法在WikiSQL和MNLI上的超参数分析。随着可训练参数数量的增加, 前缀嵌入调优 (PrefixEmbed) 和前缀层调优 (PrefixLayer) 的表现都变差, 而LoRA的表现稳定。性能通过验证准确率来衡量。

Method	MNLI(m)-100	MNLI(m)-1k	MNLI(m)-10k	MNLI(m)-392K
GPT-3 (Fine-Tune)	60.2	85.8	88.9	89.5
GPT-3 (PrefixEmbed)	37.6	75.2	79.5	88.6
GPT-3 (PrefixLayer)	48.3	82.5	85.9	89.6
GPT-3 (LoRA)	63.8	85.6	89.2	91.7

表16: 使用GPT-3 175B在MNLI子集上不同方法的验证准确率。MNLI- n 描述了一个包含 n 训练样本的子集。我们使用完整的验证集进行评估。与包括微调在内的其他方法相比, LoRA在样本效率上表现良好。

具体来说, 设 $U_A^{i\top} U_B^j$ 的奇异值为 $\sigma_1, \sigma_2, \dots, \sigma_p$, 其中 $p = \min\{i, j\}$ 。我们知道投影度量 Ham & Lee (2008) 定义为:

$$d(U_A^i, U_B^j) = \sqrt{p - \sum_{i=1}^p \sigma_i^2} \in [0, \sqrt{p}]$$

Hyperparameters	Adaptation	MNLI-100	MNLI-1k	MNLI-10K	MNLI-392K
Optimizer	-			AdamW	
Warmup Tokens	-			250,000	
LR Schedule	-			Linear	
Batch Size	-	20	20	100	128
# Epoch	-	40	40	4	2
Learning Rate	FineTune			5.00E-6	
	PrefixEmbed	2.00E-04	2.00E-04	4.00E-04	5.00E-04
	PrefixLayer	5.00E-05	5.00E-05	5.00E-05	1.00E-04
	LoRA			2.00E-4	
Adaptation-Specific	PrefixEmbed l_p	16	32	64	256
	PrefixEmbed l_i			8	
	PrefixTune		$l_p = l_i = 8$		
	LoRA		$r_q = r_v = 8$		

Table 17: The hyperparameters used for different GPT-3 adaptation methods on MNLI(m)- n .

where our similarity is defined as:

$$\phi(A, B, i, j) = \psi(U_A^i, U_B^j) = \frac{\sum_{i=1}^p \sigma_i^2}{p} = \frac{1}{p} \left(1 - d(U_A^i, U_B^j)^2 \right)$$

This similarity satisfies that if U_A^i and U_B^j share the same column span, then $\phi(A, B, i, j) = 1$. If they are completely orthogonal, then $\phi(A, B, i, j) = 0$. Otherwise, $\phi(A, B, i, j) \in (0, 1)$.

H ADDITIONAL EXPERIMENTS ON LOW-RANK MATRICES

We present additional results from our investigation into the low-rank update matrices.

H.1 CORRELATION BETWEEN LoRA MODULES

See Figure 6 and Figure 7 for how the results presented in Figure 3 and Figure 4 generalize to other layers.

H.2 EFFECT OF r ON GPT-2

We repeat our experiment on the effect of r (Section 7.2) in GPT-2. Using the E2E NLG Challenge dataset as an example, we report the validation loss and test metrics achieved by different choices of r after training for 26,000 steps. We present our result in Table 18. The optimal rank for GPT-2 Medium is between 4 and 16 depending on the metric used, which is similar to that for GPT-3 175B. Note that the relationship between model size and the optimal rank for adaptation is still an open question.

H.3 CORRELATION BETWEEN W AND ΔW

See Figure 8 for the normalized subspace similarity between W and ΔW with varying r .

Note again that ΔW does not contain the top singular directions of W , since the similarity between the top 4 directions in ΔW and the top-10% of those in W barely exceeds 0.2. This gives evidence that ΔW contains those “task-specific” directions that are otherwise *not* emphasized in W .

An interesting next question to answer, is how “strong” do we need to amplify those task-specific directions, in order for the model adaptation to work well?

Hyperparameters	Adaptation	MNLI-100	MNLI-1k	MNLI-10K	MNLI-392K
Optimizer	-			AdamW	
Warmup Tokens	-			250,000	
LR Schedule	-			Linear	
Batch Size	-	20	20	100	128
# Epoch	-	40	40	4	2
Learning Rate	FineTune			5.00E-6	
	PrefixEmbed	2.00E-04	2.00E-04	4.00E-04	5.00E-04
	PrefixLayer	5.00E-05	5.00E-05	5.00E-05	1.00E-04
	LoRA			2.00E-4	
Adaptation-Specific	PrefixEmbed l_p	16	32	64	256
	PrefixEmbed l_i			8	
	PrefixTune		$l_p = l_i = 8$		
	LoRA		$r_q = r_v = 8$		

表17: 在MNLI(m)-n上用于不同GPT-3适应方法的超参数。

我们的相似性定义为:

$$\phi(A, B, i, j) = \psi(U_A^i, U_B^j) = \frac{\sum_{i=1}^p \sigma_i^2}{p} = \frac{1}{p} \left(1 - d(U_A^i, U_B^j)^2 \right)$$

这种相似性满足以下条件: 如果 U_A^i 和 U_B^j 拥有相同的列跨度, 那么 $\phi(A, B, i, j) = 1$ 。如果它们完全正交, 那么 $\phi(A, B, i, j) = 0$ 。否则, $\phi(A, B, i, j) \in (0, 1)$ 。

H 关于低秩矩阵的附加实验

我们展示了我们对低秩更新矩阵研究的更多结果。

H.1 LORA 模块之间的相关性

参见图6和图7, 了解图3和图4所呈现的结果如何推广到其他层。

H.2 r 对 GPT-2 的影响

我们在 GPT-2 上重复了关于 r (第 7.2 节) 的实验。以 E2E NLG Challenge 数据集为例, 我们报告了在训练 26,000 步后, 不同 r 选择所达到的验证损失和测试指标。我们的结果展示在表 18 中。GPT-2 Medium 的最佳秩取决于所使用的指标, 介于 4 到 16 之间, 这与 GPT-3 17 5B 的情况相似。请注意, 模型规模与适应的最佳秩之间的关系仍是一个未解的问题。

H.3 W 与 ΔW 的相关性

见图8 对于 W 和 ΔW 之间的归一化子空间相似度 w 具有不同的 r 。

再次注意, ΔW 不包含 W 的最主要奇异方向, 因为 ΔW 中前四个方向与 W 中前10%的方向之间的相似度仅略高于0.2。这表明 ΔW 包含那些“任务特定”的方向, 而这些方向在 W 中本来 *not* 被强调。

接下来一个有趣的问题是, 为了使模型适应工作得好, 我们需要将那些任务特定的指令放大到多“强”的程度?

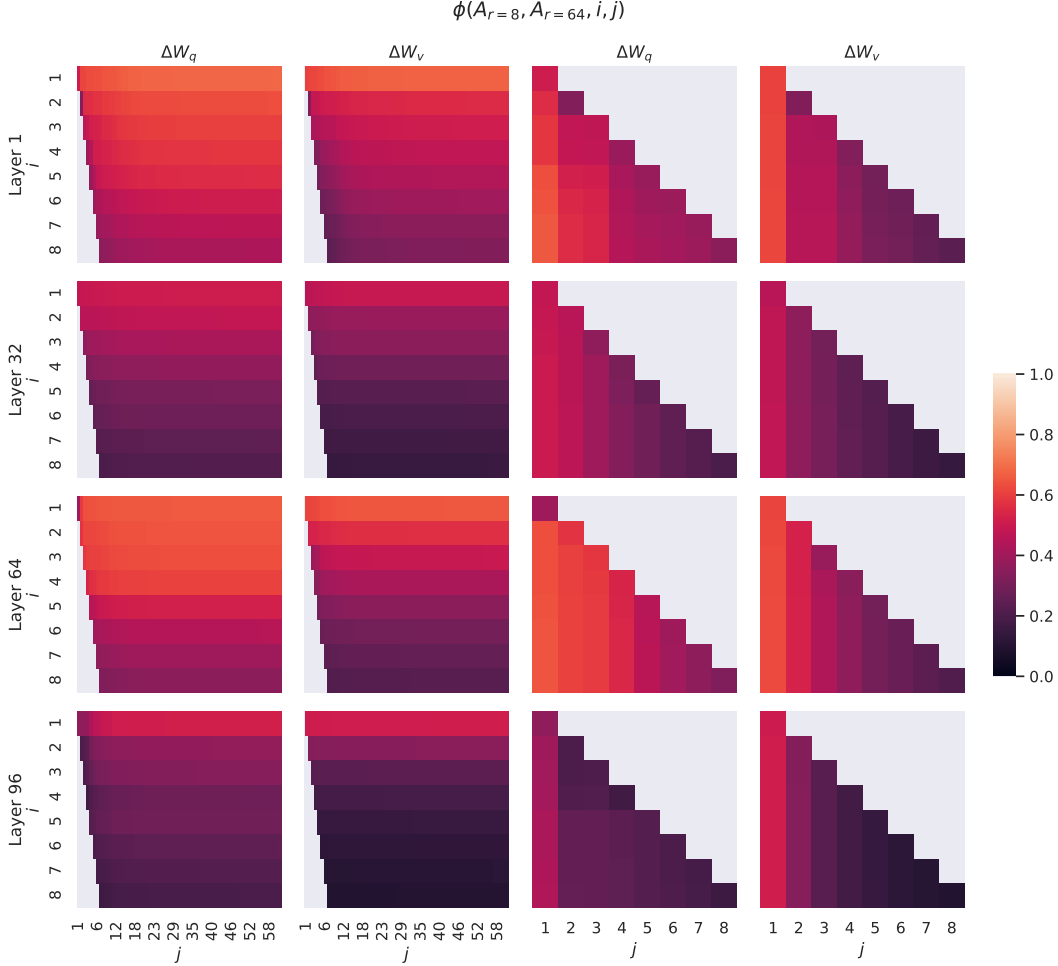


Figure 6: Normalized subspace similarity between the column vectors of $A_{r=8}$ and $A_{r=64}$ for both ΔW_q and ΔW_v from the 1st, 32nd, 64th, and 96th layers in a 96-layer Transformer.

H.4 AMPLIFICATION FACTOR

One can naturally consider a *feature amplification factor* as the ratio $\frac{\|\Delta W\|_F}{\|U^\top W V^\top\|_F}$, where U and V are the left- and right-singular matrices of the SVD decomposition of ΔW . (Recall $U U^\top W V^\top V$ gives the “projection” of W onto the subspace spanned by ΔW .)

Intuitively, when ΔW mostly contains task-specific directions, this quantity measures how much of them are amplified by ΔW . As shown in Section 7.3, for $r = 4$, this amplification factor is as large as 20. In other words, there are (generally speaking) four feature directions in each layer (out of the entire feature space from the pre-trained model W), that need to be amplified by a very large factor 20, in order to achieve our reported accuracy for the downstream specific task. And, one should expect a very different set of feature directions to be amplified for each different downstream task.

One may notice, however, for $r = 64$, this amplification factor is only around 2, meaning that *most* directions learned in ΔW with $r = 64$ are *not* being amplified by much. This should not be surprising, and in fact gives evidence (once again) that the intrinsic rank *needed* to represent the “task-specific directions” (thus for model adaptation) is low. In contrast, those directions in the rank-4 version of ΔW (corresponding to $r = 4$) are amplified by a much larger factor 20.

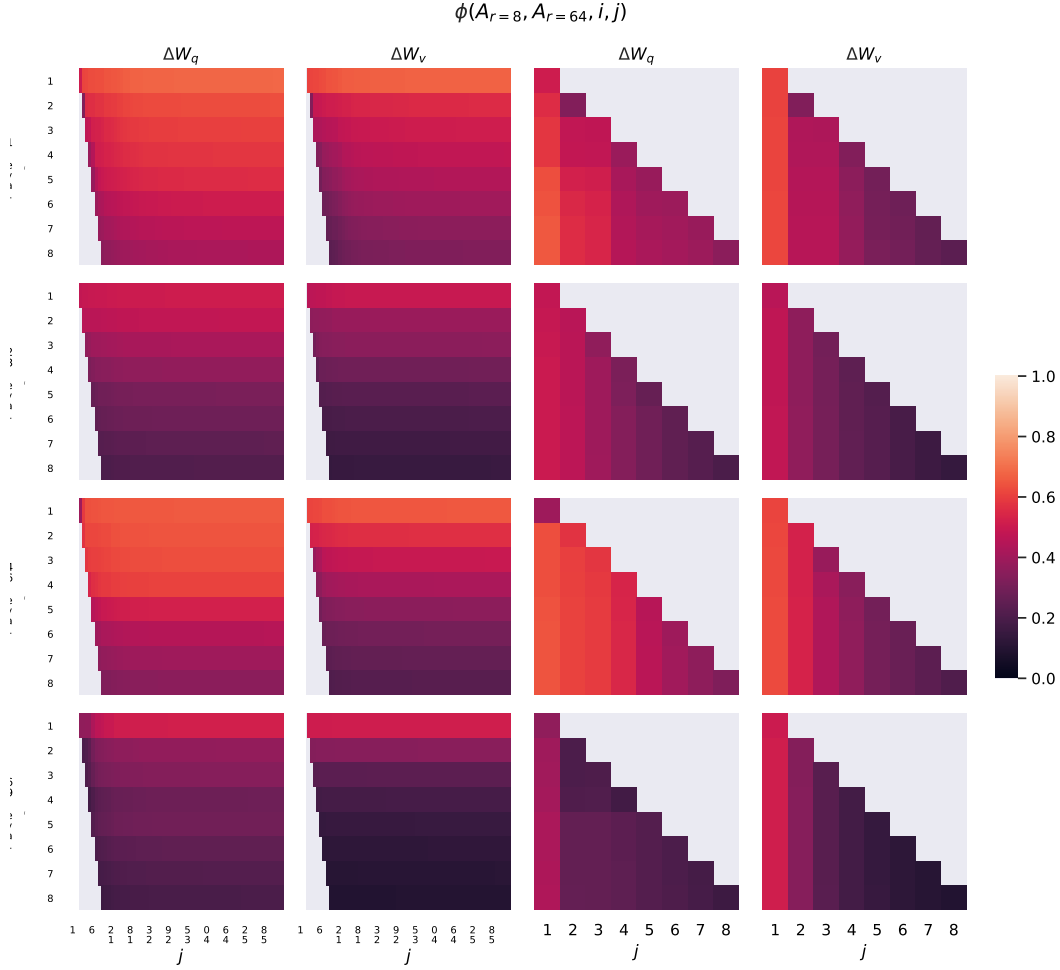


图6: 在96层Transformer的第1层、第32层、第64层和第96层中, ΔW_q 和 ΔW_v 的列向量 $A_{r=8}$ 与 $A_{r=64}$ 之间的归一化子空间相似性。

H.4 放大因子

人们可以自然地将 *feature amplification factor* 视为 $\text{ratio } \frac{\|\Delta W\|_F}{\|U^\top W V^\top\|_F}$, 其中 U 和 V 是 ΔW 的 SVD 分解的左奇异矩阵和右奇异矩阵。(回想一下, $U U^\top W V^\top V$ 给出了 W 在由 ΔW 张成的子空间上的“投影”。)

直观地说, 当 ΔW 主要包含任务特定的方向时, 这个量度衡量了它们被 ΔW 放大的程度。如第7.3节所示, 对于 $r = 4$, 这个放大因子高达20。换句话说, 一般来说, 每一层中有四个特征方向 (在整个来自预训练模型 W 的特征空间中) 需要被放大一个非常大的因子20, 以实现我们报告的下游特定任务的准确率。而且, 对于每个不同的下游任务, 应该期望放大的特征方向集合会大不相同。

然而, 人们可能会注意到, 对于 $r = 64$, 这个放大因子仅约为 2, 这意味着在 ΔW 中使用 $r = 64$ 学到的 *most* 方向 *not* 并没有被大幅放大。这并不令人惊讶, 实际上再次证明了内在秩 *needed* 来表示“任务特定方向”(因此用于模型适应)是低的。相比之下, 在 ΔW (的秩为 4 的版本中, 对应于 $r = 4$) 的那些方向被放大了更大的因子 20。

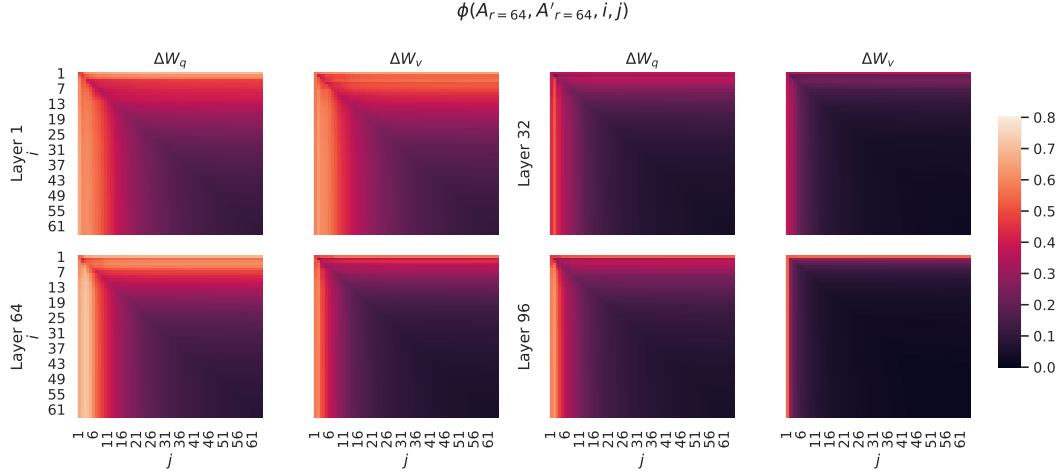


Figure 7: Normalized subspace similarity between the column vectors of $A_{r=64}$ from two randomly seeded runs, for both ΔW_q and ΔW_v from the 1st, 32nd, 64th, and 96th layers in a 96-layer Transformer.

Rank r	val_loss	BLEU	NIST	METEOR	ROUGE_L	CIDEr
1	1.23	68.72	8.7215	0.4565	0.7052	2.4329
2	1.21	69.17	8.7413	0.4590	0.7052	2.4639
4	1.18	70.38	8.8439	0.4689	0.7186	2.5349
8	1.17	69.57	8.7457	0.4636	0.7196	2.5196
16	1.16	69.61	8.7483	0.4629	0.7177	2.4985
32	1.16	69.33	8.7736	0.4642	0.7105	2.5255
64	1.16	69.24	8.7174	0.4651	0.7180	2.5070
128	1.16	68.73	8.6718	0.4628	0.7127	2.5030
256	1.16	68.92	8.6982	0.4629	0.7128	2.5012
512	1.16	68.78	8.6857	0.4637	0.7128	2.5025
1024	1.17	69.37	8.7495	0.4659	0.7149	2.5090

Table 18: Validation loss and test set metrics on E2E NLG Challenge achieved by LoRA with different rank r using GPT-2 Medium. Unlike on GPT-3 where $r = 1$ suffices for many tasks, here the performance peaks at $r = 16$ for validation loss and $r = 4$ for BLEU, suggesting the GPT-2 Medium has a similar intrinsic rank for adaptation compared to GPT-3 175B. Note that some of our hyperparameters are tuned on $r = 4$, which matches the parameter count of another baseline, and thus might not be optimal for other choices of r .

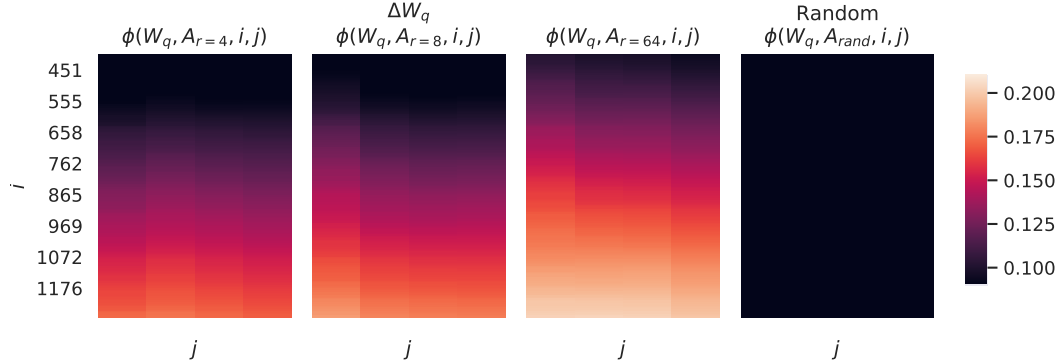


Figure 8: Normalized subspace similarity between the singular directions of W_q and those of ΔW_q with varying r and a random baseline. ΔW_q amplifies directions that are important but not emphasized in W . ΔW with a larger r tends to pick up more directions that are already emphasized in W .

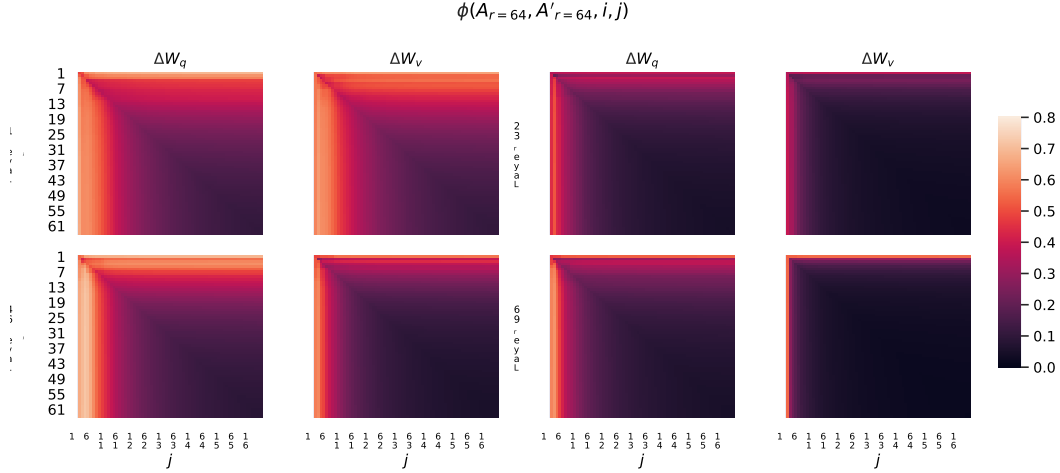


图 7: 在 96 层 Transformer 中, 对于第 1、32、64 和 96 层的 ΔW_q 和 ΔW_v , 来自两次随机种子运行的 $A_{r=64}$ 列向量之间的归一化子空间相似度。

Rank r	val_loss	BLEU	NIST	METEOR	ROUGE.L	CIDEr
1	1.23	68.72	8.7215	0.4565	0.7052	2.4329
2	1.21	69.17	8.7413	0.4590	0.7052	2.4639
4	1.18	70.38	8.8439	0.4689	0.7186	2.5349
8	1.17	69.57	8.7457	0.4636	0.7196	2.5196
16	1.16	69.61	8.7483	0.4629	0.7177	2.4985
32	1.16	69.33	8.7736	0.4642	0.7105	2.5255
64	1.16	69.24	8.7174	0.4651	0.7180	2.5070
128	1.16	68.73	8.6718	0.4628	0.7127	2.5030
256	1.16	68.92	8.6982	0.4629	0.7128	2.5012
512	1.16	68.78	8.6857	0.4637	0.7128	2.5025
1024	1.17	69.37	8.7495	0.4659	0.7149	2.5090

表 18: LoRA 使用不同秩 r 在 GPT-2 Medium 上达成的 E2E NLG 挑战的验证损失和测试集指标。与在 GPT-3 上的情况不同, $r = 1$ 对许多任务已足够, 这里验证损失的性能在 $r = 16$ 达到峰值, 而 BLEU 分数在 $r = 4$ 达到峰值, 这表明 GPT-2 Medium 在适应性方面具有与 GPT-3 175B 类似的内在秩。请注意, 我们的一些超参数是在 $r = 4$ 上调优的, 该值与另一个基线的参数数量相匹配, 因此可能并不适用于其他 r 的选择。

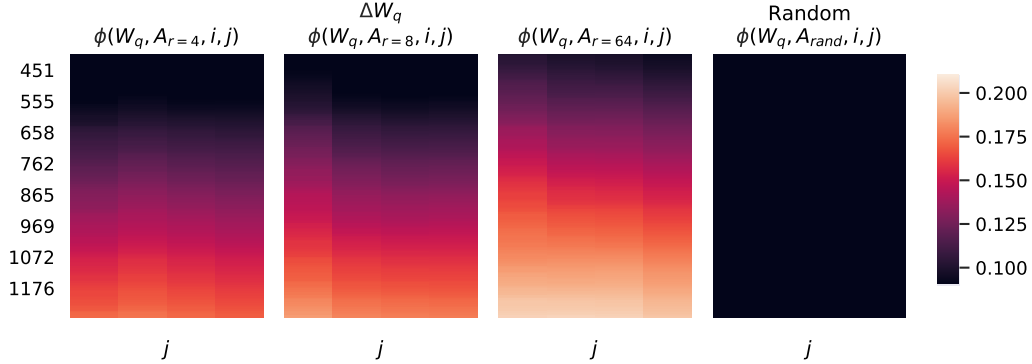


图8: W_q 的奇异方向与 ΔW_q 的奇异方向在不同 r 和随机基线下的归一化子空间相似度。 ΔW_q 放大了在 W 中重要但未被强调的方向。具有较大 r 的 ΔW 往往会捕捉到在 W 中已被强调的更多方向。